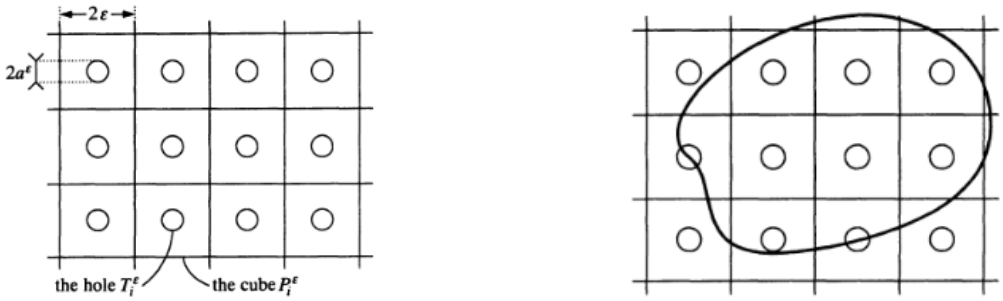
Projet d'Application Recherche n°153
Restitution

Un terme étrange venu d'ailleurs
Introduction à l'homogénéisation

— Rapport final —



(a) (b)

Petites boules réparties périodiquement dans $\mathbb{R}^n$
(Source : [1])

Présidente du Jury
Cécile Nouguier

Elèves

Jules Bougnol
Maxime Luce

Equipe pédagogique

Matthieu Bonnivard (tuteur)
Olivier Dessombz (conseiller)

Année 2025/2026 – 19 avril 2026

Résumé

Ce rapport de projet d'application de recherche présente une étude détaillée de la théorie de l'homogénéisation appliquée à la résolution d'équations aux dérivées partielles (problème de Dirichlet) dans des domaines périodiquement perforés. L'objectif principal est de comprendre, d'expliciter et de vérifier l'apparition d'un « terme étrange venu d'ailleurs » lors du changement d'échelle mathématique, en s'appuyant sur l'article de Cioranescu et Murat [1].

Notre méthodologie s'articule autour de deux axes : une étude théorique à partir du livre de H. Brezis et de l'article de Cioranescu et Murat visant à redémontrer la valeur théorique de ce terme correctif en dimension deux $(N = 2)$ et une validation empirique réalisée via la méthode des éléments finis à l'aide du logiciel FreeFem++.

Pour pallier d'importants problèmes de convergence initiaux, un algorithme de sélection stricte des paramètres de simulation a été développé sous Matlab, permettant d'optimiser le maillage et de faire chuter l'erreur relative à environ 1%.

Enfin, notre travail s'est concentré sur la dimension 3 (forêts de cylindres et de sphères). Cette nouvelle variante du problème a mis en évidence des défis numériques tels que la saturation mémoire ou la taille du maillage, qui ont été étudiés et résolus notamment par l'implémentation d'une méthode de pénalisation.

Finalement, ce travail présente succinctement les modèles et résultats mathématiques nécessaires pour établir une méthodologie numérique fiable pour l'analyse de tels problèmes multi-échelles.

Abstract

This research application project report presents a detailed study of homogenization theory applied to the resolution of partial differential equations (Dirichlet problem) in periodically perforated domains. The main objective is to understand, to make explicit, and to verify the appearance of a "strange term coming from elsewhere" during the mathematical scaling process, based on the article by Cioranescu and Murat [1].

Our methodology is structured around two axes : a theoretical study based on the book by H. Brezis and the article by Cioranescu and Murat, aiming to re-derive the theoretical value of this corrective term in dimension two $(N = 2)$, and an empirical validation carried out via the finite element method using the FreeFem++ software.

To overcome significant initial convergence problems, an algorithm for strict selection of simulation parameters was developed in Matlab, making it possible to optimize the mesh and reduce the relative error to about 1%.

Finally, our work focused on dimension 3 (forests of cylinders and spheres). This new variant of the problem highlighted numerical challenges such as memory saturation or mesh size, which were studied and resolved in particular through the implementation of a penalization method.

Ultimately, this work briefly presents the mathematical models and results necessary to establish a reliable numerical methodology for the analysis of such multi-scale problems.

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes et organisations qui ont contribué, de près ou de loin, à la réussite de notre projet.

Tout d'abord, un immense merci à M. Bonnivard pour son investissement, son encadrement, son enseignement, sa disponibilité et ses nombreux retours constructifs. C'est grâce à lui que nous avons pu nous initier à l'homogénéisation et que nous avons pu continuer à prendre plaisir à faire des mathématiques.

Nos remerciements vont aussi à notre conseiller, M. Dessombz, pour son accompagnement attentif tout au long du projet. Nous le remercions pour nos échanges lors de nos différentes rencontres qui nous ont permis d'approfondir et de questionner notre maîtrise du sujet.

Nous remercions finalement l'école Centrale de Lyon, le laboratoire Camille Jordan et toutes les personnes ayant permis la réalisation de ce projet de recherche qui nous a tant apporté scientifiquement.

Table des matières

Résumé **1**

Abstract **2**

Remerciements **3**

1 Contexte et problématique **5**
1.1 Un terme étrange 5
1.2 Objectifs 6

2 État de l'art **8**
2.1 Présentation générale 8
2.2 Méthodologie de recherche 8
2.2.1 Définition de l'homogénéisation 8
2.2.2 Définition de la méthode des éléments finis 8
2.2.3 Implémentation pratique de la méthode des éléments finis 9
2.2.4 Choix du logiciel : FreeFem++ 9
2.3 Contexte historique 9
2.4 Synthèse des résultats mathématiques du Brezis 10

3 Résultats mathématiques et numériques **12**
3.1 Trous sphériques distribué périodiquement dans le domaine 12
3.1.1 Un résultat important de l'article 12
3.1.2 Problème considéré 14
3.1.3 Cas $N = 2$ 15
3.1.3.1 Définition de w^ε 15
3.1.3.2 Calcul de μ 17
3.2 Vérification numérique des résultats de l'article 22
3.2.1 Élaboration du code FreeFEM : observations des problèmes de convergence 22
3.2.2 Choix des paramètres 25
3.2.3 Choix du maillage 26
3.2.4 Optimisation du maillage et choix des paramètres 27
3.2.5 Comparaison des champs résultants 28
3.2.6 Cas $N = 3$ 31
3.3 Perspective d'évolution 35

Conclusion **37**

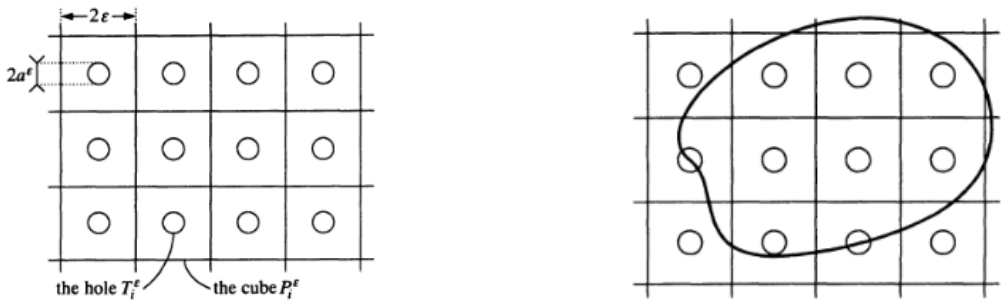
Annexes **39**
Présentation des annexes 39
Codes implémentés 39
Code MATLAB pour déterminer les triplets 39
Code FreeFEM final pour le cas 2D 41
Code FreeFEM final pour les cylindres 46
Code FreeFEM final pour les sphères 51

1 Contexte et problématique

1.1 Un terme étrange

L'homogénéisation est au cœur de la résolution de problèmes physiques très divers tels que l'étude des matériaux composites. En effet, on peut représenter ces matériaux par des milieux parcourus périodiquement (en première approximation) par des fibres. En projetant dans le plan, il s'agit alors d'un domaine perforés périodiquement. En s'intéressant aux comportements de ces matériaux, on est amenés à résoudre des équations aux dérivées partielles qui ne possèdent pas de solution directe.

L'homogénéisation consiste alors à changer d'espace en remplaçant le problème initial par une équation « effective » décrivant le comportement macroscopique du milieu. Notre Projet d'Application Recherche (PAr) vise à étudier une des applications de l'homogénéisation présentée dans l'article de Cioranescu et Murat [1]. Dans cet article, il est notamment question de résoudre un problème de Dirichlet dans le domaine perforé présenté sur la Figure 2.



(a) (b)

Figure 2 – Petites boules réparties périodiquement dans $\mathbb{R}^n$, exemple de l'article de Cioranescu et Murat (Source : [1])

Il est alors possible de montrer que le problème se réécrit avec l'apparition d'un « terme étrange venu d'ailleurs », noté μ, dont on se propose de vérifier la valeur numérique, puisqu'elle est le résultat d'un calcul de limite que nous détaillerons dans notre synthèse de l'article 3.1. Pour comprendre et retrouver les expressions théoriques de μ, il a été préalablement nécessaire de découvrir et d'étudier les rudiments des espaces topologiques faibles et de l'analyse fonctionnelle dans un cadre plus général 3.1.

Comment redémontrer ce « terme étrange venu d'ailleurs » mathématiquement puis comment le retrouver empiriquement par simulation informatique ?

Nous rappelons d'abord les objectifs fixés lors du *référentiel de gestion de projet* (RGP) 1.2 avant de présenter notre état de l'art 2 principalement constitué de la synthèse des résultats nécessaires à la compréhension de l'article de Cioranescu et Murat tirés du livre *Analyse fonctionnelle* de H. Brezis [2].

Nous terminons par présenter le travail réalisé 3 avec, notamment, la synthèse de l'article de Cioranescu et Murat et les démonstrations des expressions théoriques de μ dans le cas particulier $N = 2$. Nous terminons ensuite en présentant notre implémentation numérique permettant de retrouver la convergence en fonction des paramètres du problèmes vers la valeur théorique de μ.

1.2 Objectifs

L'objet de ce *projet d'application de recherche* est de répondre à ces objectifs :

- découvrir ce qu'est l'homogénéisation et se familiariser avec les outils mathématiques associés ;
- étudier en détails une section d'un article de recherche pour comprendre comment un tel article est structuré et identifier les éléments importants de démonstrations ;
- mettre en application ces connaissances en travaillant spécifiquement sur un exemple et développer un code *éléments finis*.

Afin de réaliser ces objectifs généraux, nous avons identifié des objectifs spécifiques et chronologiques :

1) acquérir des connaissances sur l'approche variationnelle du problème de Dirichlet ;
2) comprendre les éléments principaux de la première section de l'article *"A Strange Term Coming from Nowhere"* et l'exemple 2.1 ;
3) proposer une méthode numérique permettant de valider les résultats obtenus dans l'article ;
4) utiliser ce code pour étudier d'autres configurations, c'est-à-dire "explorer d'autres termes venu d'ailleurs".

Le Tableau 1 résume ces quatre objectifs avec le détails de chaque objectif, la priorité associée par rapport à la réussite globale du projet ; les indicateurs de réussite et livrables et enfin l'évaluation de l'avancée.

Objectif	**Détails de l'objectif**	**Priorité**	**Livrable/Indicateur de réussite**	**Évaluation de l'avancée**
Acquérir des connaissances sur l'approche variationnelle du problème de Dirichlet	Étudier le livre de H. Brezis "Analyse fonctionnelle" [2] en particulier : la topologie (faible notamment) ; les espaces L^p ; les espaces de Hilbert ; espaces de Sobolev (cas 1D et dimension N)	**Élevée** puisqu'elle conditionne notre compréhension	Réalisation d'un document de synthèse des résultats importants ; Point régulier avec notre tuteur pour consolider notre apprentissage	Quels chapitres/-résultats avons-nous traité ?
Article "A Strange Term Coming from Nowhere"	A la lumière des connaissances et compétences acquises, on peut lire et étudier la première section de l'article de recherche et détailler en particulier les calculs de l'exemple 2.1.	**Élevée** puisque l'implémentation en éléments finis se basera sur notre compréhension de l'exemple et des résultats de l'article	Production d'un résumé des résultats permettant d'aboutir au « terme étrange venu de nulle part » ; Réunion avec notre tuteur	Avancée dans la première section de l'article et étude des résultats de l'exemple 2.1
Proposer une méthode numérique permettant de valider les résultats obtenus dans l'article	Étudier la méthode des éléments finis et proposer une implémentation permettant de retrouver le résultat de l'article. Apprendre à utiliser *FreeFem++*.	**Critique** car il s'agit du cœur de notre projet d'application recherche	Avoir un code fonctionnel pour le cas traité dans l'exemple 2.1 de l'article	Première étape de modélisation et d'apprentissage de l'utilisation de *FreeFem++* ; Seconde étape d'implémenta-tion
Explorer d'autres termes venu d'ailleurs	Envisager d'autres cas d'études : d'autres géométries, d'autres conditions aux limites	**Faible**	Réaliser un code modulaire permettant plusieurs études	Modéliser et implémenter la modularité nécessaire

Tableau 1 – Tableau de synthèse des objectifs : détails, priorité, indicateurs de réussite et évaluation de l'avancée

2 État de l'art

2.1 Présentation générale

Suite aux échanges avec notre tuteur, une synthèse exhaustive de l'homogénéisation n'a pas été retenu puisque peu pertinente par rapport à notre sujet. Il nous a plutôt conseillé de découvrir l'homogénéisation à travers deux références principales :

- le livre *Analyse fonctionnelle* de H. Brezis [2] pour nous permettre d'acquérir les notions mathématiques nécessaires au bon déroulé du projet ;
- l'article dont le projet fait l'objet : *"A Strange Term Coming from Nowhere"* [1].

Ce projet a pour objet l'homogénéisation restreinte à l'application proposée dans l'article de Cioranescu et Murat.

Les problématiques majeures étaient donc d'aborder de nouvelles notions en analyse fonctionnelle et de découvrir une application par le prisme d'un article de recherche.

2.2 Méthodologie de recherche

Cette sous-section vise à présenter le travail en définissant les concepts utilisés au cours de ce projet. Il introduit de plus les outils utilisés pour la recherche d'informations, avec une explication du choix de ces outils.

2.2.1 Définition de l'homogénéisation

L'étude des phénomènes physiques consiste bien souvent, après une première modélisation des équations régissant le phénomène étudié, en une approche numérique nécessaire à la résolution de cesdites équations. Cependant, il n'est pas rare que dans certains cas la puissance de calcul ne soit pas suffisante devant les caractéristiques intrinsèques complexes du problème étudié, notamment lorsque ce dernier présente deux échelles de dimensions caractéristiques très différentes, faisant ainsi considérablement varier les propriétés locales du milieu.

L'homogénéisation propose ainsi une approche systématique à ce genre de situations en remplaçant le problème initial présentant deux échelles très différentes par un problème "effectif" ou "homogénéisé" décrivant le comportement de la solution dans le cas où l'échelle la plus faible tend vers zéro. Cette méthode fournit donc une équation alternative à celle de départ, et la remplace par un problème macroscopique équivalent plus simple et dont la résolution numérique est cette fois-ci bien moins coûteuse.

2.2.2 Définition de la méthode des éléments finis

La méthode des éléments finis est une méthode de discrétisation des EDP adaptée aux problèmes qui peuvent se poser sous une forme dite variationnelle. Celle-ci consiste à écrire le problème à résoudre de la manière suivante (1) avec V un espace de Hilbert, $B : V \times V \to \mathbb{R}$ une forme bilinéaire continue et coercive et $l : V \to \mathbb{R}$ une forme linéaire continue (d'après [3] et [4]).

$$\forall v \in V, \quad B[u, v] = l(v) \tag{1}$$

Ces hypothèses permettent d'appliquer le théorème de Lax-Milgram (ou de Riesz si B est de plus symétrique) qui permet de s'assurer de l'unicité de la solution.

A l'inverse de la méthode des différences finies qui discrétise l'EDP elle-même, la méthode des éléments finis a pour objet la discrétisation de la formulation variationnelle du problème (1).

En pratique, il va donc s'agir de remplacer par V par V_h un sous-espace vectoriel de V dans l'équation (1) qui devient (2), c'est la méthode de Galerkin [3].

$$\forall v \in V_h, \quad B[u_h, v_h] = l(v_h) \tag{2}$$

Concrètement, les sous espaces V_h seront obtenus à partir du domaine spatial Ω, en le discrétisant, tel que présenté dans l'exemple précédent en mécanique des solides avec le maillage.

2.2.3 Implémentation pratique de la méthode des éléments finis

En pratique, la plupart des logiciels se décomposent en trois majeures parties [5] :

- le préprocesseur, c'est la partie dans laquelle on
  - définit la géométrie de notre problème ;
  - renseigne les paramètres du modèle ;
  - définit le maillage ;
  - applique les conditions aux limites.
- la solution, dans laquelle on
  - choisit le type d'analyse ;
  - renseigne les options des algorithmes de résolution ;
  - lance la phase de résolution.
- le postprocesseur dans lequel on va analyser et traiter les données pour déterminer les quantités d'intérêt.

2.2.4 Choix du logiciel : FreeFem++

FreeFem++ est un des logiciels permettant de résoudre des problèmes variationnels par la méthode des éléments finis en deux ou trois dimensions. Notre tuteur nous a recommandé ce logiciel, qui a en plus l'avantage d'être open-source et d'avoir une documentation fournie.

2.3 Contexte historique

Le contexte applicatif de notre projet est celui des matériaux composites. Historiquement, l'analyse de ces matériaux a débuté avec des estimations empiriques, notamment étudiées en cours d'IDM [6], avec les lois de mélange de Voigt et Reuss. Celles-ci permettent d'estimer les rigidités par le truchement des fractions volumiques des constituants. Des modèles plus avancés ont vu le jour dans

les années 1960 avec notamment les travaux d'Eshelby sur les inclusions [7] avec pour la première fois des considérations énergétiques.

Cependant, l'essor des matériaux composites à hautes performances des années 1970-1980 nécessitait des modèles plus précis. Cet essor est motivé par les excellentes caractéristiques mécaniques et le bon compromis avec la masse rendu possible par la présence de fibre au sein d'une matrice. Ces estimations plus précises ont été rendues possibles par l'application de la théorie de l'homogénéisation, théorie notamment exposée dans [2].

La méthode des éléments finis a enfin permis de considérer des problèmes d'une complexité plus grande (tissage en 3D des fibres, comme illustré dans l'article de Cioranescu et Murat, ou encore les nanocomposites) et avec des modèles non-linéaires. C'est cette raison qui a motivé ce PAr : comprendre l'apport fondamental dans la résolution de problèmes mathématiques (et donc physiques) de l'homogénéisation et de la méthode des éléments finis conjointement.

2.4 Synthèse des résultats mathématiques du Brezis

Notre synthèse des informations scientifiques nécessaire à la compréhension du sujet a été réalisée à partir du livre *Analyse fonctionnelle* de H. Brezis [2] suivant les conseils de notre tuteur. Cette synthèse s'appuie également sur les cours : Outils mathématiques avancés pour l'analyse des équations aux dérivées partielles dispensé par Laurent Seppecher à centrale Lyon, ainsi que les cours de Licence 3 suivis à l'université Claude Bernard Lyon 1.

On présente ici succinctement les résultats incontournables du livre et de ces cours pour rendre accessible notre travail sur l'article. La synthèse complète se trouve en Annexes.

Définition 1. Convergence faible — Une suite $(u_n)_{n\in\mathbb{N}}$ **converge faiblement** vers $u \in E$ si et seulement si

$$\forall f \in E', \qquad \langle f, u_n\rangle_{E',E} \to \langle f, u\rangle_{E',E}$$

Propriété 1. Propriétés de la convergence faible — Soit $(u_n)_{n\in\mathbb{N}}$ une suite de E. On a

1) Si $u_n \to u$ fortement, alors $u_n \rightharpoonup u$ faiblement.
2) Si $u_n \rightharpoonup u$ alors $(u_n)_{n\in\mathbb{N}}$ est bornée et $\|u\| \leq \liminf \|u_n\|$.
3) Si $u_n \rightharpoonup u$ et si $f_n \rightharpoonup f$ alors $\langle f_n, u_n\rangle_{E',E} \to \langle f, u\rangle_{E',E}$

Théorème 1. Riesz — Soit H un espace de Hilbert et H' son dual. Pour tout $L \in H'$, il existe un unique $\ell \in H$ tel que

$$Lu = \langle \ell, v\rangle_H, \qquad \forall u \in H \tag{3}$$

De plus, $\|\ell\|_H = \|L\|_{H'}$.

Corollaire 1. — Une suite (u_n) d'un espace de Hilbert H converge faiblement vers $u \in H$ si et seulement si

$$\forall v \in H, \qquad \lim_{n\to+\infty} \langle u_n, v\rangle_H = \langle u, v\rangle_H.$$

Corollaire 1. — Soit $\mathcal{D}$ une partie dense de H. Une suite bornée (u_n) de H converge faiblement vers $u \in H$ si et seulement si

$$\forall v \in \mathcal{D}, \qquad \lim_{n \to +\infty} \langle u_n, v \rangle_H = \langle u, v \rangle_H.$$

Théorème 2. Lax-Milgram — Soit H un espace de Hilbert et H' son dual. Soit $L \in H'$ et $a : H^2 \to \mathbb{R}$ une forme bilinéaire continue et coercive. Alors il existe un unique $u \in H$ tel que :

$$a(u, v) = Lv, \qquad \forall v \in H \tag{4}$$

et il existe une constante $C > 0$ qui dépend seulement de a, telle que $\|u\|_H \leq C\|L\|_{H'}$. De plus, u est l'unique minimiseur de la fonction

$$J(v) := \frac{1}{2}a(u, v) - Lv$$

.

Théorème 3. Théorème de Rellich — Si Ω est un ouvert borné régulier de classe $\mathcal{C}^1$, alors de toute suite bornée de $H^1(\Omega)$ on peut extraire une sous-suite convergente dans $L^2(\Omega)$ (on dit que l'injection canonique de $H^1(\Omega)$ dans $L^2(\Omega)$ est compacte).

3 Résultats mathématiques et numériques

Nous présentons le travail réalisé de manière chronologique :

- la présentation de l'article et les résultats importants de la première section et notre démonstration de la valeur théorique de μ dans le cas $N = 2$ 3.1 ;
- la méthode des éléments finis proposée pour retrouver cette valeur théorique par la simulation informatique 3.2.

3.1 Trous sphériques distribué périodiquement dans le domaine

3.1.1 Un résultat important de l'article

On commence par présenter la propriété qui sous-tend les résultats de l'exemple que l'on considère. Le problème général considéré pour $f \in L^2(\Omega)$ est :

$$\begin{cases} -\Delta u^\varepsilon = f \quad \text{dans} \quad D'(\Omega^2) \\ u^\varepsilon \in H_0^1(\Omega) \end{cases} \tag{5}$$

Soit Ω un ouvert borné de $\mathbb{R}^n$. On fait les hypothèses suivantes :

(H.1) $\omega^\varepsilon \in H^1(\Omega)$,
(H.2) $\omega^\varepsilon = 0$ sur les trous $T_i^\varepsilon, 1 \leq i \leq n(\varepsilon)$,
(H.3) $\omega^\varepsilon \rightharpoonup 1$ faiblement dans $H^1(\Omega)$;
(H.4) $\mu \in W^{-1,\infty}(\Omega)$;
(H.5) pour toute suite v^ε telle que $v^\varepsilon = 0$ sur $T_i^\varepsilon, 1 \leq i \leq n(\varepsilon)$, vérifiant $v^\varepsilon \rightharpoonup v$ faiblement dans $H^1(\Omega)$, on a $\langle -\Delta w^\varepsilon, \varphi v^\varepsilon \rangle_{H^{-1}(\Omega), H_0^1(\Omega)} \to \langle \mu, \varphi v \rangle_{H^{-1}(\Omega), H_0^1(\Omega)}$ pour tout $\varphi \in \mathcal{D}(\Omega)$

Propriété 2. — Si w^ε et μ vérifient les hypothèses (H. 1) à (H. 5), on a :

$$\langle \mu, \varphi \rangle = \lim_{\varepsilon \to 0} \int_\Omega |\text{grad } w^\varepsilon|^2 \, \varphi \, \mathrm{d}x$$

Remarque. — Cette proposition permet de justifier la positivité de μ et donc d'appliquer par la suite le théorème de Lax-Milgram.

Démonstration. On prend $v^\varepsilon = w^\varepsilon$ et $v = 1$ dans l'hypothèse (H. 5), ces derniers vérifient bien les hypothèses et on a donc

$$\forall \varphi \in \mathcal{D}(\Omega), \quad \langle -\Delta w^\varepsilon, \varphi w^\varepsilon \rangle_{H^{-1}(\Omega), H_0^1(\Omega)} \to \langle \mu, \varphi \rangle_{H^{-1}(\Omega), H_0^1(\Omega)} . \tag{6}$$

On peut réécrire le crochet en réalisant une intégration par parties puisqu'il n'y a pas de terme de bord puisque le domaine est à support compact. On a donc

$$\langle -\Delta w^\varepsilon, \varphi w^\varepsilon \rangle_{H^{-1}(\Omega), H_0^1(\Omega)} = \int_\Omega \text{grad } w^\varepsilon \text{grad } (\varphi w^\varepsilon) \, \mathrm{d}v$$

On applique alors la règle de Leibniz pour développer le gradient car grad $(fg) = f\text{grad } g + g\text{grad } f$. Il vient donc

$$\int_\Omega \text{grad } w^\varepsilon \text{grad } (\varphi w^\varepsilon) \, \mathrm{d}v = \int_\Omega |\text{grad } w^\varepsilon|^2 \varphi \mathrm{d}v + \int_\Omega w^\varepsilon \text{grad } w^\varepsilon \text{grad } \varphi \mathrm{d}v$$

D'après l'hypothèse (H. 3), on a

$$\lim_{\varepsilon \to 0} \text{grad } w^\varepsilon \rightharpoonup 0 \text{ dans } \left(L^2(\Omega)\right)^N$$

et le théorème de Rellich-Kondrashov permet de montrer que $w^\varepsilon \to 1$ fortement dans l'ensemble des fonctions qui sont de carré intégrable sur tout sous-ensemble compact inclus dans le domaine Ω, noté $L^2_{loq}(\Omega)$.

Ainsi,

$$\int_\Omega w^\varepsilon \text{grad } w^\varepsilon \text{grad } \varphi \mathrm{d}v \underset{\varepsilon \to 0}{\longrightarrow} 0$$

.

Ainsi,

$$\langle -\Delta w^\varepsilon, \varphi w^\varepsilon \rangle_{H^{-1}(\Omega), H_0^1(\Omega)} \underset{\varepsilon \to 0}{\longrightarrow} \int_\Omega |\text{grad } w^\varepsilon|^2 \varphi \mathrm{d}v$$

L'équation (6) permet alors de conclure que :

$$\boxed{\langle \mu, \varphi \rangle_{H^{-1}(\Omega), H_0^1(\Omega)} = \lim_{\varepsilon \to 0} \int_\Omega |\text{grad } w^\varepsilon|^2 \varphi \mathrm{d}v}$$

■

Théorème 4. — Sous les hypothèses (H. 1) à (H. 5), la solution u^ε de (5) vérifie

$$\tilde{u^\varepsilon} \rightharpoonup u \quad \text{faiblement dans } H_0^1(\Omega),$$

où u est l'unique solution du problème suivant

$$\begin{cases} -\Delta u + \mu u = f & \text{dans } \mathcal{D}'(\Omega) \\ u \in H_0^1(\Omega) \end{cases} \tag{7}$$

Remarque. — La limite u est alors la solution d'un problème de Dirichlet pour l'opérateur $-\Delta + \mu$, qui contient ce « terme étrange » μ par rapport à l'opérateur initial $-\Delta$.

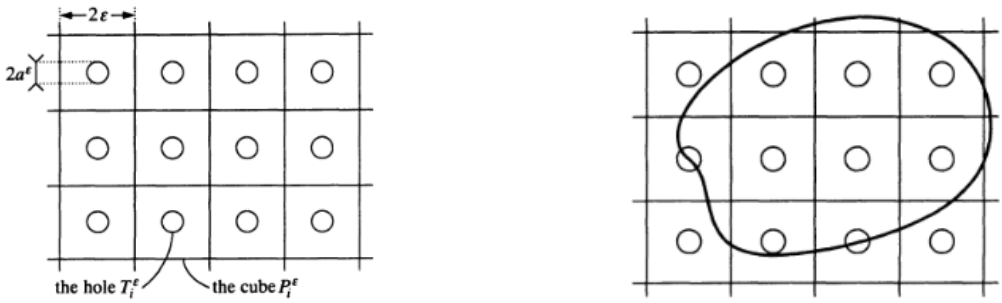
3.1.2 Problème considéré

Pour chaque valeur de ε, on recouvre $\mathbb{R}^n$ $(n \geq 2)$ de cubes, notés P_i^ε, de taille 2ε. On retire de chaque cube un « trou » de rayon a^ε avec donc $a^\varepsilon > 0$ centré. Ainsi, $\mathbb{R}^n$ est perforé périodiquement par des trous identiques. On pose alors

$$Q^\varepsilon = \mathbb{R}^n \backslash \bigcup T_i^\varepsilon,$$

$$\Omega_\varepsilon = \Omega \cap Q^\varepsilon = \Omega \backslash \bigcup_{i=1}^{n(\varepsilon)} T_i^\varepsilon,$$

où $n(\varepsilon)$ est le nombre de trous dans le domaine. Ainsi, Q^ε représente l'ensemble privé de l'ensemble des trous et Ω_e le domaine privé des trous. On représente schématiquement ces ensembles sur la Figure 3.



(a) Partie de $\mathbb{R}^n$ (b) Un exemple de domaine Ω

FIGURE 3 – Petites boules réparties périodiquement dans $\mathbb{R}^n$
(Source : [1])

On définit alors w^ε sur chaque cube P_i^ε par l'équation (8).

$$\begin{cases} w^\varepsilon & = 0 \quad \text{in } T_i^\varepsilon, \\ \Delta w^\varepsilon & = 0 \quad \text{in } B_i^\varepsilon - T_i^\varepsilon, \\ w^\varepsilon & = 1 \quad \text{in } P_i^\varepsilon - B_i^\varepsilon, \\ w^\varepsilon & \quad \text{continues sur les interfaces,} \end{cases} \tag{8}$$

où $B_i^\varepsilon \subset P_i^\varepsilon$ est le trou de rayon ε concentrique à T_i^ε (Figure 4). Cela définit donc w^ε dans $\mathbb{R}^n$ donc en particulier dans Ω. Cette fonction w^ε est périodique de période $]-\varepsilon; +\varepsilon[^N$. On ne peut cependant pas l'écrire comme $w\left(\frac{x}{\varepsilon}\right)$ sauf dans le cas particulier où l'on a $a^\varepsilon = C\varepsilon$ avec $0 < C < 1$.

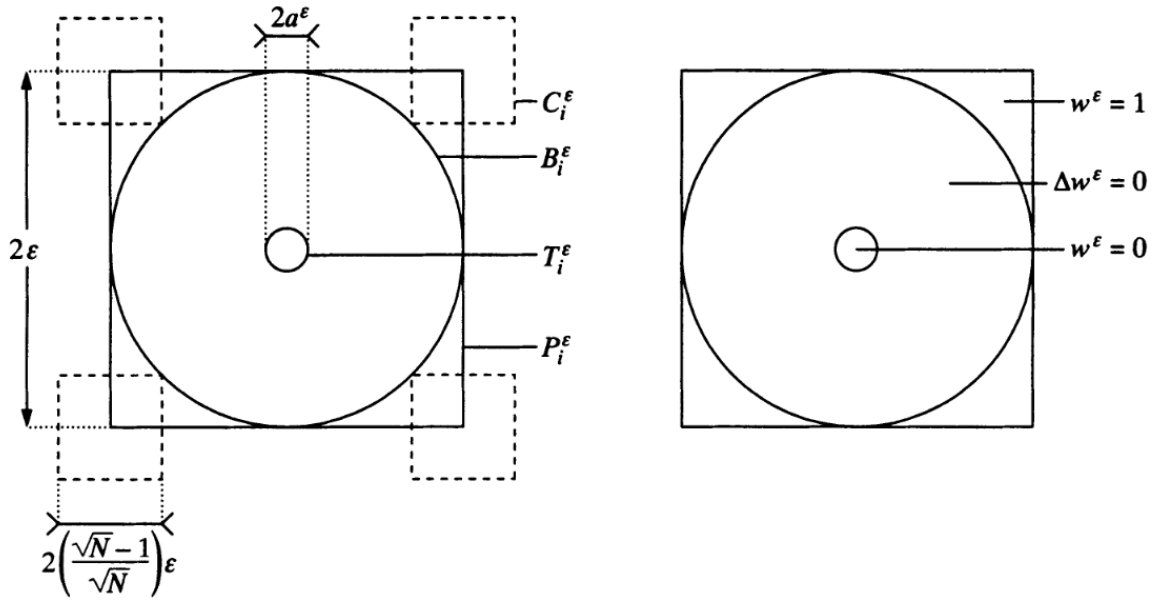


FIGURE 4 – Définition de w^ε

On peut alors déterminer w^ε en utilisant les coordonnées polaires dans l'anneau $B_i^\varepsilon - T_i^\varepsilon$. On trouve (la démonstration pour le cas $N = 2$ est faite dans la sous-section 3.1.3)

$$\begin{cases} w^\varepsilon = \frac{\ln a^\varepsilon - \ln r}{\ln a^\varepsilon - \ln \varepsilon} & \text{si } N = 2, \\ w^\varepsilon = \frac{(a^\varepsilon)^{-(N-2)} - (r)^{-(N-2)}}{(a^\varepsilon)^{-(N-2)} - (\varepsilon)^{-(N-2)}} & \text{si } N \geq 3, \end{cases} \tag{9}$$

On définit alors μ par

$$\begin{cases} \mu = \frac{\pi}{2}\frac{1}{C_0} & \text{si } N = 2, \\ \mu = \frac{S_N(N-2)}{2^N} C_0^{N-2} & \text{si } N \geq 3, \end{cases} \tag{10}$$

avec S_N la surface de la sphère unité dans $\mathbb{R}^n$ et C_0 une constante réelle strictement positive.

Théorème 5. — Si on prend

$$\begin{cases} a^\varepsilon = \exp\left(-\frac{C_0}{\varepsilon^2}\right) & \text{si } N = 2, \\ a^\varepsilon = C_0 \varepsilon^{N/(N-2)} & \text{si } N \geq 3, \end{cases} \tag{11}$$

alors w^ε et μ défini par les équations (4) et (10) satisfont les hypothèses (H. 1) à (H. 5).

3.1.3 Cas $N = 2$

3.1.3.1 Définition de w^ε

Propriété 3. — Pour le cas $N = 2$, pour respecter la définition de w^ε, on a

$$w^\varepsilon = \frac{\ln a^\varepsilon - \ln r}{\ln a^\varepsilon - \ln^\varepsilon}$$

Démonstration.

w^ε est défini par (8). On se place en coordonnées polaires dont l'expression du Laplacien pour une fonction f quelconque est

$$\Delta f = \frac{1}{r}\frac{\partial}{\partial r}\left(r\frac{\partial f}{\partial r}\right) + \frac{1}{r^2}\frac{\partial^2 f}{\partial \theta^2} + \frac{\partial^2 f}{\partial z^2}$$

Le problème considéré possède une invariance selon θ et z, on a donc $w^\varepsilon(r,\theta,z) = w^\varepsilon(r)$. On se place dans un cube P_i^ε. Ainsi, w^ε vérifie plus précisément :

$$\begin{cases} w^\varepsilon = 0 & \text{si } r = a^\varepsilon, \\ w^\varepsilon = 1 & \text{si } r = \varepsilon, \\ \Delta w^\varepsilon = 0 & \text{si } a^\varepsilon \leq r \leq \varepsilon \end{cases} \tag{12}$$

Soit $r \in [a^\varepsilon; \varepsilon]$, il vient alors en notant K_1 et K_2 deux constantes :

$$\begin{aligned} \Delta w^\varepsilon(r) = 0 &\Longrightarrow \frac{1}{r}\frac{\partial}{\partial r}\left(r\frac{\partial w^\varepsilon}{\partial r}(r)\right) = 0 \\ &\Longrightarrow r\frac{\partial w^\varepsilon}{\partial r}(r) = K_1 \quad (\text{car } r \neq 0) \\ &\Longrightarrow w^\varepsilon(r) = K_1 \ln(r) + K_2 \end{aligned}$$

On applique enfin les conditions aux bords de l'anneau (12) et il vient :

$$\begin{cases} K_1 \ln(a^\varepsilon) + K_2 & = 0 \\ K_1 \ln(\varepsilon) + K_2 & = 1 \end{cases} \quad \Longrightarrow \quad \begin{cases} K_1 & = \frac{1}{\ln(\varepsilon) - \ln(a^\varepsilon)} \\ K_2 & = \frac{\ln(a^\varepsilon)}{\ln(a^\varepsilon) - \ln(\varepsilon)} \end{cases}$$

Et donc finalement, il vient :

$$\boxed{w^\varepsilon = \frac{\ln a^\varepsilon - \ln r}{\ln a^\varepsilon - \ln^\varepsilon}}$$

■

3.1.3.2 Calcul de μ

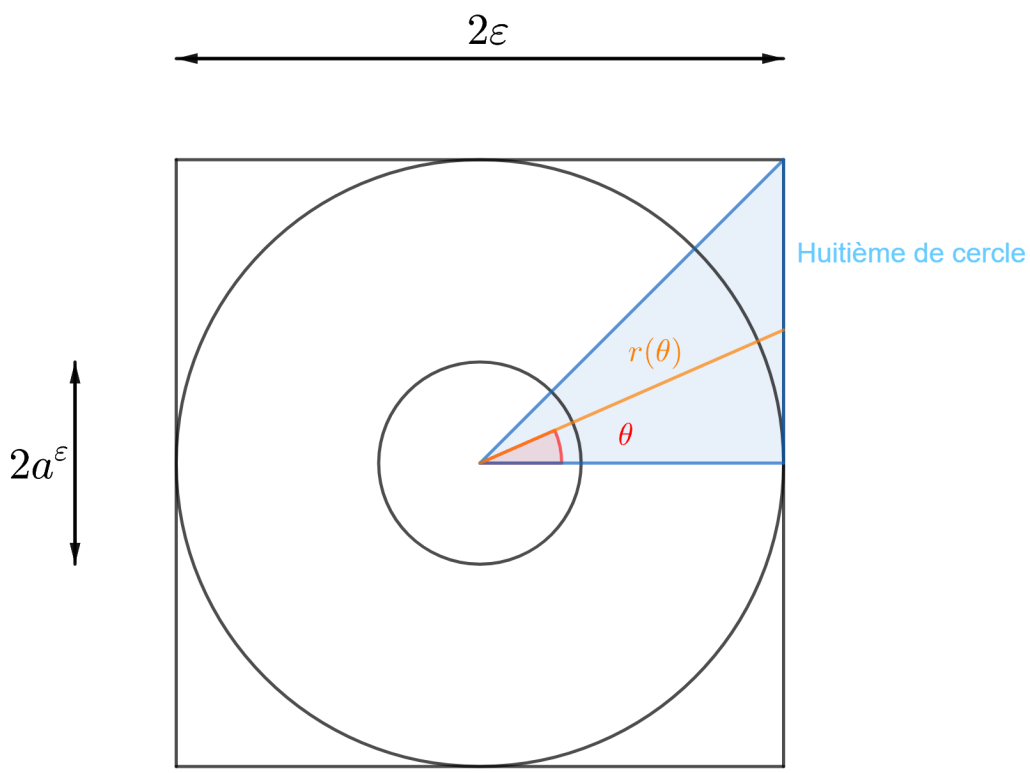


FIGURE 5 – Schématisation du problème et coordonnées cylindriques

Dans le cas $N = 2$, on a :

$$\begin{cases} \omega^\varepsilon = \frac{\ln(a^\varepsilon)-\ln(r)}{\ln(a^\varepsilon)-\ln(\varepsilon)} \\ a^\varepsilon = \exp\left(-\frac{C_0}{\varepsilon^2}\right) \end{cases} \tag{13}$$

Propriété 4. — Pour le cas $N = 2$, on a :

$$\mu = \frac{\pi}{2C_0}$$

Démonstration.

La détermination du terme μ dans le cas où $N = 2$ repose essentiellement sur le calcul de l'intégrale suivante

$$I = \int_{P_i^\varepsilon} |\text{grad}\,(w^\varepsilon)|^2 \mathrm{d}x$$

En considérant la géométrie de P_i^ε illustrée sur la Figure 5, cette intégrale peut se réécrire sous la forme

$$
\begin{aligned}
I &= 8\int_{\theta=0}^{\theta=\frac{\pi}{4}}\int_{r=a_\varepsilon}^{r=\frac{\varepsilon}{\cos(\theta)}}\left(\frac{1}{\frac{C_0}{\varepsilon^2}+\ln(\varepsilon)}\right)^2\frac{1}{r}\mathrm{d}r\mathrm{d}\theta \\
&= 8\int_{\theta=0}^{\theta=\frac{\pi}{4}}\left(\frac{1}{\frac{C_0}{\varepsilon^2}+\ln(\varepsilon)}\right)^2\left(\ln\left(\frac{\varepsilon}{\cos(\theta)}\right)-\ln(a_\varepsilon)\right)\mathrm{d}\theta \\
&= 8\int_{\theta=0}^{\theta=\frac{\pi}{4}}\left(\frac{1}{\frac{C_0}{\varepsilon^2}+\ln(\varepsilon)}\right)^2\left(\ln\left(\frac{\varepsilon}{\cos(\theta)}\right)+\frac{C_0}{\varepsilon^2}\right)\mathrm{d}\theta \\
&= 8\left(\frac{1}{\frac{C_0}{\varepsilon^2}+\ln(\varepsilon)}\right)^2\left(\int_{\theta=0}^{\theta=\frac{\pi}{4}}\ln\left(\frac{\varepsilon}{\cos(\theta)}\right)\mathrm{d}\theta+\frac{\pi}{4}\frac{C_0}{\varepsilon^2}\right)
\end{aligned}
$$

Afin d'obtenir l'expression exacte de I, il convient de calculer l'intégrale présente dans l'expression précédente.

$$J = \int_0^{\frac{\pi}{4}}\ln\left(\frac{\varepsilon}{\cos(\theta)}\right)\mathrm{d}\theta$$

D'abord, on peut simplifier l'intégrale de la façon suivante :

$$J = \int_0^{\frac{\pi}{4}}\ln(\varepsilon)\mathrm{d}\theta - \int_0^{\frac{\pi}{4}}\ln\left(\cos(\theta)\right)\mathrm{d}\theta$$

D'une part, la première intégrale est : $\int_0^{\frac{\pi}{4}}\ln(\varepsilon)\mathrm{d}\theta = \frac{\pi}{4}\ln(\varepsilon)$.

Pour la seconde, on pose $A = \int_0^{\frac{\pi}{4}}\ln\left(\cos(\theta)\right)\mathrm{d}\theta$ et $B = \int_0^{\frac{\pi}{4}}\ln\left(\sin(\theta)\right)\mathrm{d}\theta$.

Pour déterminer A et B, on calcule séparément $A+B$ et $A-B$.

- **Calcul de $A+B$:** On a :

$$A+B = \int_0^{\frac{\pi}{4}}\ln\left(\cos(\theta)\right)+\ln\left(\sin(\theta)\right)\mathrm{d}\theta = \int_0^{\frac{\pi}{4}}\ln\left(\cos(\theta)\sin(\theta)\right)\mathrm{d}\theta$$

Or, $\sin\theta\cos\theta = \frac{\sin(2\theta)}{2}$ donc il vient :

$$A+B = \int_0^{\frac{\pi}{4}}\ln\left(\frac{\sin(2\theta)}{2}\right)\mathrm{d}\theta = \int_0^{\frac{\pi}{4}}\ln\left(\sin(2\theta)\right)\mathrm{d}\theta - \int_0^{\frac{\pi}{4}}\ln(2)\,\mathrm{d}\theta$$

On pose alors $u = 2\theta$ (changement de variable de classe $\mathcal{C}^1$ et bijectif) donc il vient successivement :

$$\int_0^{\frac{\pi}{4}} \ln\left(\sin(2\theta)\right) \mathrm{d}\theta = \frac{1}{2}\int_0^{\frac{\pi}{2}} \ln\left(\sin u\right) \mathrm{d}u$$

Or, en posant $x = \frac{\pi}{2} - u$, il vient :

$$\int_0^{\frac{\pi}{2}} \ln\left(\sin u\right) \mathrm{d}u = \int_0^{\frac{\pi}{2}} \ln\left(\sin x\right) \mathrm{d}x = E$$

On a alors :

$$\begin{aligned}
2E &= \int_0^{\frac{\pi}{2}} \ln\left(\sin u\right) \mathrm{d}u + \int_0^{\frac{\pi}{2}} \ln\left(\sin x\right) \mathrm{d}x \\
&= \int_0^{\frac{\pi}{2}} \ln\left(\sin x \cos x\right) \mathrm{d}x \\
&= \int_0^{\frac{\pi}{2}} \ln\left(\frac{\sin(2x)}{2}\right) \mathrm{d}x \qquad \text{(par duplication)} \\
&= \int_0^{\frac{\pi}{2}} \ln\left(\sin 2x\right) \mathrm{d}x - \int_0^{\frac{\pi}{2}} \ln\left(2\right) \mathrm{d}x \\
&= \frac{1}{2}\int_0^{\pi} \ln\left(\sin(u)\right) \mathrm{d}u - \frac{\pi}{2}\ln(2) \qquad \text{(en posant } u = 2x\text{)}
\end{aligned}$$

Or par symétrie du sinus sur $[0, \pi]$, il vient :

$$\frac{1}{2}\int_0^{\pi} \ln\left(\sin(u)\right) \mathrm{d}u = \frac{1}{2}\left(2\int_0^{\frac{\pi}{2}} \ln\left(\sin(u)\right)\right) \mathrm{d}u = E$$

On remplace alors dans l'égalité précédente et il vient finalement :

$$2E = E - \frac{\pi}{2}\ln(2) \Longrightarrow \boxed{E = \int_0^{\frac{\pi}{2}} \ln\left(\sin u\right) \mathrm{d}u = -\frac{\pi}{2}\ln(2)}$$

On peut alors substituer cette valeur dans l'expression de $A + B$ et il vient finalement :

$$A + B = \frac{1}{2}E - \frac{\pi}{4}\ln(2) \Longrightarrow \boxed{A + B = -\frac{\pi}{2}\ln(2)}$$

— **Calcul de $A - B$:** On a :

$$A - B = \int_0^{\frac{\pi}{4}} \ln(\cos(\theta)) - \ln(\sin(\theta))\,\mathrm{d}\theta = \int_0^{\frac{\pi}{4}} \ln\left(\frac{\cos(\theta)}{\sin(\theta)}\right)\mathrm{d}\theta = \int_0^{\frac{\pi}{4}} \ln(\cot\theta)\,\mathrm{d}\theta$$

Or, on a le lien entre cette intégrale et le nombre de Catalan :

$$\int_0^{\frac{\pi}{4}} \ln(\cot\theta)\,\mathrm{d}\theta = G$$

où G désigne la constante de Catalan définie par :

$$G = \sum_{n=0}^{+\infty} \frac{(-1)^n}{(2n+1)^2} = 1 - \frac{1}{3^2} + \frac{1}{5^2} - \frac{1}{7^2} + \ldots$$

En effet, commençons tout d'abord par simplifier l'intégrale précédente (notée K dans toute la suite) en notant que $\cot(\theta) = \frac{1}{\tan(\theta)}$ et donc :

$$\ln(\cot(\theta)) = -\ln(\tan(\theta))$$

Le changement de variable $x = \tan(\theta)$ donne alors

$$K = \int_0^{\frac{\pi}{4}} \ln(\cot\theta)\,\mathrm{d}\theta = \int_0^1 -\frac{\ln(x)}{1+x^2}\mathrm{d}x$$

Par ailleurs le développement en série entière de $\frac{1}{1+x^2}$ s'écrit

$$\frac{1}{1+x^2} = \sum_{n=0}^{+\infty} (-1)^n x^{2n}$$

En réinjectant cette expression dans l'intégrale ci-dessus, il vient

$$K = \int_0^1 \left(-\ln(x) \sum_{n=0}^{+\infty} (-1)^n x^{2n}\right)\mathrm{d}x$$

Or la série converge uniformément sur tout compact de $[0, 1[$ et l'intégrale est bien définie, ainsi le théorème d'intégration terme à terme permet d'intervertir somme et intégrale, soit

$$K = \sum_{n=0}^{+\infty} (-1)^{n+1} \int_0^1 \ln(x) x^{2n}\mathrm{d}x$$

Notons alors K_n le $n^{\text{ième}}$ terme de cette somme. L'expression de K_n est alors donnée par

$$K_n = (-1)^{n+1} \int_0^1 \ln(x) x^{2n}\mathrm{d}x$$

Il est possible de calculer cette expression à l'aide d'une intégration par parties, on obtient

$$\begin{aligned}
K_n &= (-1)^{n+1}\left(\int_0^1 \ln(x)x^{2n}\mathrm{d}x\right)\\
&= (-1)^{n+1}\left(\left[\frac{x^{2n+1}}{2n+1}ln(x)\right]_0^1 - \int_0^1 \frac{x^{2n+1}}{2n+1}\frac{1}{x}\mathrm{d}x\right)\\
&= (-1)^n \int_0^1 \frac{x^{2n}}{2n+1}\mathrm{d}x\\
&= (-1)^n \frac{1}{(2n+1)^2}
\end{aligned}$$

D'où *in fine*,

$$\boxed{\int_0^{\frac{\pi}{4}} \ln\left(\cot\theta\right)\mathrm{d}\theta = \sum_{n=0}^{+\infty}\frac{(-1)^n}{(2n+1)^2} = G}$$

Il vient donc finalement :

$$\boxed{B - A = G}$$

— Bref, on a :

$$\begin{cases} A+B &= -\frac{\pi}{2}\ln 2\\ A-B &= G\end{cases} \Longrightarrow \begin{cases} A &= -\frac{\pi}{4}\ln(2)+\frac{G}{2}\\ B &= -\frac{\pi}{4}\ln(2)-\frac{G}{2}\end{cases} \Longrightarrow \boxed{\int_0^{\frac{\pi}{4}} \ln\left(\cos(\theta)\right)\mathrm{d}\theta = -\frac{\pi}{4}\ln(2)+\frac{G}{2}}$$

En injectant dans l'expression de J, il vient :

$$J = \frac{\pi}{4}\left(\ln(\varepsilon)+\ln(2)-\frac{4}{\pi}\frac{G}{2}\right)$$

Et donc :

$$\begin{aligned}
I &= 8\left(\frac{1}{\frac{C_0}{\varepsilon^2}+\ln(\varepsilon)}\right)^2\left(\frac{\pi}{4}\left(\ln(\varepsilon)+\ln(2)-\frac{4}{\pi}\frac{G}{2}\right)+\frac{\pi}{4}\frac{C_0}{\varepsilon^2}\right)\\
&= 8\left(\frac{1}{\frac{C_0}{\varepsilon^2}+\ln(\varepsilon)}\right)^2\frac{\pi}{4}\left(\ln(\varepsilon)+\frac{C_0}{\varepsilon^2}+\ln(2)-\frac{4}{\pi}\frac{G}{2}\right)
\end{aligned}$$

On note c la constante $\ln(2)-\frac{4}{\pi}\frac{G}{2}$ et on a :

$$I = 8\left(\frac{1}{\frac{C_0}{\varepsilon^2} + \ln(\varepsilon)}\right)^2 \frac{\pi}{4}\left(\ln(\varepsilon) + \frac{C_0}{\varepsilon^2} + c\right)$$

Ainsi, il vient :

$$\begin{aligned}
\mu &= \lim_{\varepsilon\to 0} \frac{I}{(2\varepsilon)^2} \\
&= \lim_{\varepsilon\to 0} \frac{1}{4\varepsilon^2}\left[8\left(\frac{1}{\frac{C_0}{\varepsilon^2} + \ln(\varepsilon)}\right)^2 \frac{\pi}{4}\left(\ln(\varepsilon) + \frac{C_0}{\varepsilon^2} + c\right)\right] \\
&= \lim_{\varepsilon\to 0} \frac{2\pi}{\frac{C_0}{\varepsilon^2} + \ln(\varepsilon)} \frac{1}{4\varepsilon^2} \\
&= \lim_{\varepsilon\to 0} \frac{\pi}{2} \frac{1}{C_0 + \ln(\varepsilon)\varepsilon^2} \\
&\boxed{\mu = \frac{\pi}{2C_0}}
\end{aligned}$$

■

Remarque. — On retrouve ce résultat en intégrant uniquement sur la couronne entre $r = a^\varepsilon$ et $r = \varepsilon$!

3.2 Vérification numérique des résultats de l'article

3.2.1 Élaboration du code FreeFEM : observations des problèmes de convergence

Une fois le cadre théorique de l'homogénéisation clairement établi, et le cas $N = 2$ traité du point de vu théorique, l'enjeu consiste à valider les résultats obtenus par une approche numérique. Pour ce faire, en accord avec ce qui a été indiqué précédemment, nous utilisons le logiciel FreeFem, particulièrement adapté à la résolution de ce type de problème, en effet, ce langage possède une structure très proche du langage mathématique, rendant la transcription de problème plus simple qu'avec d'autres langages de programmation. En exploitant la périodicité du problème, et comme cela a été rappelé avant, il est possible de se restreindre à l'étude d'une cellule. Ainsi, l'implémentation se décompose ainsi globalement en deux phases : une première phase de définition du problème englobant à la fois la description géométrique, la construction du maillage et les conditions aux limites et une seconde phase de résolution lors de laquelle les calculs sont menés et les comparaisons sont effectuées afin de vérifier les résultats de la partie précédente. Le détails du code implémenté est donné ci-dessous.

La première étape consiste à introduire les paramètres définissant les dimensions caractéristiques du problème.

```
real epsilon = 2.3357;
real aEpsilon = 0.83252;
real C0 = -epsilon^2*log(aEpsilon);
```

Listing 1 – Définition des paramètres du problème

Il est alors également possible de construire le maillage utilisé dans la suite. Pour ce faire, on définit les éléments géométriques, ici les cercles *CircleOut* et *CircleIn*.

```
// labels
int labSquare = 1;       // Bord du cube P^epsilon
int labCircleOut = 2;    // Bord du cercle B^epsilon
int labHole = 3;         // Bord du trou T^epsilon

// densite du maillage
int nin = 100;
int nout=nin*epsilon/aEpsilon;

// definition des frontieres
// le cercle exterrieur
border CircleOut(t=0, 2*pi) { x = bEpsilon*cos(t); y = bEpsilon*sin(t);
    label = labCircleOut; }
// le trou
border CircleIn(t=0, 2*pi)  { x = aEpsilon*cos(t); y = aEpsilon*sin(t);
    label = labHole; }

// domaine : le carre avec le cercle privé du trou
mesh Th = buildmesh(
    CircleOut(nout)+
    CircleIn(-nin)//- n car trou
);
```

Listing 2 – Définition des bordures de la cellule et du maillage

Le signe "-" attribué à la densité de points du cercle intérieur permet la création du trou, c'est à dire d'un vide. De plus pour maintenir une densité de points cohérente entre les deux cercles, nous introduisons le paramètre `nout = nin * epsilon / aEpsilon`.

Note : Une fois le maillage généré, la commande `real h = Th.hmax;` permet de récupérer la taille maximale des éléments. Cela permettra par la suite de tracer des courbes de convergence de l'erreur en fonction de h afin de valider l'ordre de la méthode des éléments finis.

Une fois le problème posé, il convient de résoudre le problème introduit précédemment. Pour se faire, on commence par résoudre numériquement une équation de Laplace $\Delta w = 0$ sur le domaine perforé T_ε. Les conditions de Dirichlet de ce problème sont $w = 0$ sur le bord du trou et $w = 1$ sur le bord du cercle extérieur.

```
// resolution probleme local
fespace Vh(Th, P2); // Éléments finis quadratiques
Vh w, v;

// Problème variationnel : Laplacien(w) = 0

solve ProbW(w, v) =
    int2d(Th)(
        dx(w)*dx(v) + dy(w)*dy(v)
    ) // grad de w . grad de v
    + on(labHole, w = 0) // w = 0 sur le trou
    + on(labCircleOut, w = 1); // w = 1 sur le cercle
```

Listing 3 – Résolution de l'équation de Laplace

La constante d'homogénéisation numérique μ_{num} est ensuite extraite via le calcul de l'énergie de Dirichlet du système présenté plus tôt.

```
// terme etrange venu d'ailleurs

real Energy = int2d(Th)(dx(w)^2 + dy(w)^2); // on integre sur le domaine le grad au carre
real VolumeCellule = (2*epsilon)^2; // on calcule le volume de la cellule

// on fait le rapport pour determiner mu_numerique
real MuNum = Energy / VolumeCellule;

// expression de mu theorique pour N=2
real MuTh = (pi/2)*1/C0 ;

// on compare avec mu calcule avec w_epsilon theorique
func r = sqrt(x^2 + y^2);
func wFormula = (log(aEpsilon) - log(r)) / (log(aEpsilon) - log(epsilon));

Vh wTh = wFormula;  // on projette sur Vh pour calculer ensuite dx() et dy()

// calcul mu theorique 2
real MuTh2 = int2d(Th)( (dx(wTh)^2 + dy(wTh)^2) ) / VolumeCellule;
//real MuTh2 = int2d(Th)( (dx(wTh)^2 + dy(wTh)^2) * (r <= epsilon) ) / VolumeCellule;
// *(r <= epsilon) pour integrer que sur l'anneau A VERIFIER

real I = (2*pi)/(C0/epsilon^2 + log(epsilon));
real muTheorique3 = I/((2*epsilon)^2);
```

Listing 4 – Calcul de la constante d'homogénéisation et comparaison avec la théorie

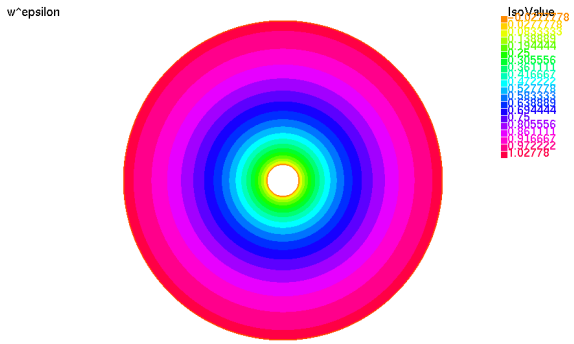
Une comparaison est ensuite effectuée avec la valeur limite théorique établie afin de valider ce résultat.

Note : Afin de s'assurer que l'erreur ne provienne pas de l'une des différentes étape intermédiaire (intégration numérique ...), on effectue systématiquement des comparaisons intermédiaires.

Les premiers choix naïfs de paramètres conduisent néanmoins à des résultats peu concluant.

```
--- Parametres ---
Epsilon   = 0.1
a^epsilon = 0.01
b^epsilon = 0.0995
C0 = 0.0460517
  --  mesh:  Nb of Triangles = 176386, Nb of Vertices 88743
  -- Solve :
          min 9.48562e-33  max 1
-----------------------------------------------
Resultats :
Energie calculee (int |grad w|^2) : 2.73433
Mu Numerique (Energie/Vol)       : 68.3582
Mu Theorique pi/(2*C0)         : 34.1094
Erreur relative Numerique/Theorique                : 100.409 %
MuTh2 (Integrale wTh)  = 68.08
Erreur relative Numerique/Theorique2                : 0.408623 %
MuTh3 : 68.2188
Erreur relative Numerique/Theorique3                : 0.204254 %
-----------------------------------------------
times: compile 0.124s, execution 16.989s,  mpirank:0
```

(a) Jeu de paramètres et erreurs relatives obtenues



(b) Isovaleurs de w

FIGURE 6 – Résultats obtenus pour $\varepsilon = 0.1$ et $a_\varepsilon = 0.01$

Bien que les paramètres respectent a priori les conditions imposées par la théorie, à savoir $C_0 = \varepsilon^2 \ln(a_\varepsilon)$, et que ε est faible, on trouve ici une erreur de 100%. Cet élément peut toutefois s'expliquer par le calcul mené dans la partie 3.1.3 ayant conduit à la formule suivante :

$$\mu = \lim_{\varepsilon \to 0} \frac{\pi}{2} \frac{1}{C_0 + \ln(\varepsilon)\varepsilon^2}$$

La convergence vers $\frac{\pi}{2C_0}$ est alors obtenue si les conditions précédentes sont remplies et $C_0 >> \varepsilon^2 \ln(\varepsilon)$ c'est-à-dire $\varepsilon^2 \ln(a_\varepsilon) >> \varepsilon^2 \ln(\varepsilon)$. La partie suivante s'intéresse donc à la prise en compte de cette condition supplémentaire et donc à la détermination de triplets $(\varepsilon, a_\varepsilon, C_0)$ pour lesquels la convergence est observable.

3.2.2 Choix des paramètres

L'obtention de triplets $(\varepsilon, a_\varepsilon, C_0)$ permettant d'observer la convergence nécessite donc une étude supplémentaire. A cet effet, un code MATLAB a été implémenté pour prendre en compte le problème du rapport entre $\varepsilon^2 \ln(a_\varepsilon)$ et $\varepsilon^2 \ln(\varepsilon)$. Par ailleurs, la sélection des triplets doit également prendre en compte des éléments propres à FREEFEM ainsi qu'aux choix ayant été effectués pour le maillage : la valeur de a_ε ne doit pas être inférieure à 10^{-6} (valeur fixée par FREEFEM) et le rapport entre a_ε et ε doit être tel que le nombre de points sur le cercle extérieur pour la maillage soit inférieur à 2000 (valeur également fixée par FREEFEM).

Pour valider les résultats obtenus avec FREEFEM, nous avons implémenté un script MATLAB permettant de tracer l'évolution de l'erreur.

Les configurations obtenues (dont une liste exhaustive est donnée dans le Tableau 2) permettent d'aboutir à des résultats bien meilleurs comme le montre la figure ci-contre.

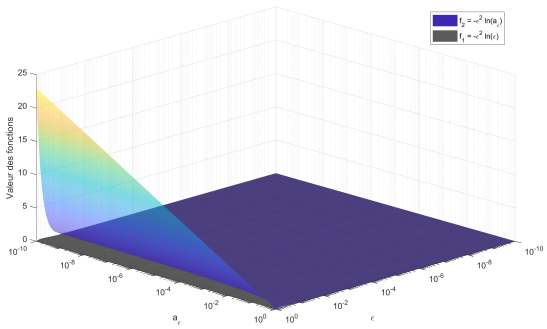
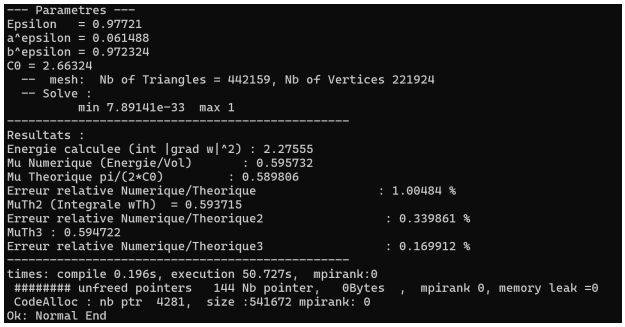


FIGURE 7 – Comparaison 3D de $f_1(\varepsilon)$ et $f_2(\varepsilon, a_\varepsilon)$

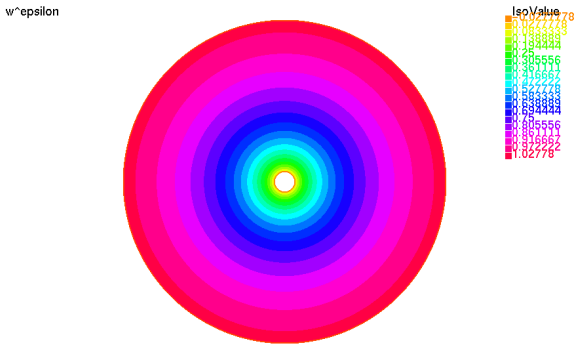
ε	a_ε	C_0	n_{out}
0.97051	0.048626	-2.8479	1996
0.97949	0.11038	-2.1143	888
0.98401	0.13394	-1.9466	735
0.98855	0.065145	-2.669	1518
0.99083	0.10231	-2.2382	969
0.99312	0.12791	-2.0282	777
0.9954	0.12703	-2.0444	784
0.9977	0.10044	-2.2876	994
1	0.063369	-2.7588	1579
1	0.63384	-0.45596	158

TABLEAU 2 – Liste exhaustive de triplets sélectionnés par le code Matlab



```
--- Parametres ---
Epsilon   = 0.97721
a^epsilon = 0.061488
b^epsilon = 0.972324
C0 = 2.66324
  --  mesh:  Nb of Triangles = 442159, Nb of Vertices 221924
  -- Solve :
          min 7.89141e-33  max 1
-----------------------------------------------
Resultats :
Energie calculee (int |grad w|^2) : 2.27555
Mu Numerique (Energie/Vol)        : 0.595732
Mu Theorique pi/(2*C0)            : 0.589806
Erreur relative Numerique/Theorique                  : 1.00484 %
MuTh2 (Integrale wTh)  = 0.593715
Erreur relative Numerique/Theorique2                  : 0.339861 %
MuTh3 : 0.594722
Erreur relative Numerique/Theorique3                  : 0.169912 %
-----------------------------------------------
times: compile 0.196s, execution 50.727s,  mpirank:0
 ######## unfreed pointers   144 Nb pointer,   0Bytes  ,  mpirank 0, memory leak =0
 CodeAlloc : nb ptr  4281,  size :541672 mpirank: 0
Ok: Normal End
```

(a) Jeu de paramètres et erreurs relatives obtenues



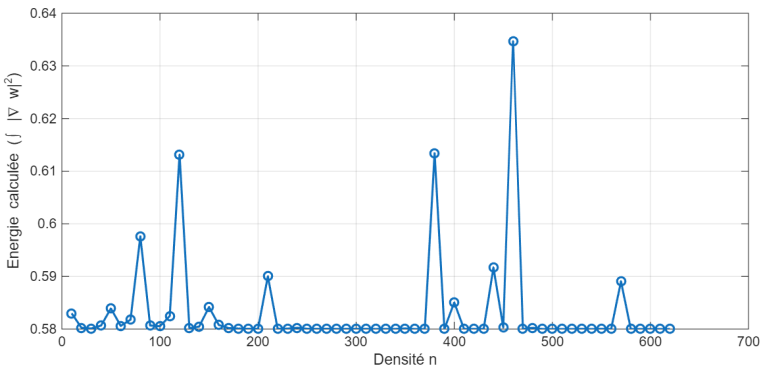
(b) Isovaleurs de w

FIGURE 8 – Résultats obtenus pour $\varepsilon = 0.1$ et $a_\varepsilon = 0.01$

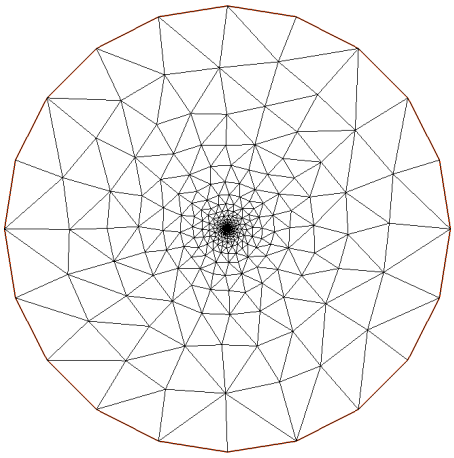
On obtient cette fois-ci une erreur relative de l'ordre de 1% entre les valeurs théorique et numérique, la convergence semble bien se produire. Cependant afin de conclure définitivement sur l'observation de la convergence, une étude supplémentaire est menée afin de visualiser les impacts respectifs de la convergence vers 0 de ε et de la densité de maillage choisie pour l'étude. Ces deux facteurs joueront un rôle fondamental dans l'étude d'autres configurations pour lesquelles une formule théorique n'existe pas nécessairement.

3.2.3 Choix du maillage

Nous avons commencé par attribuer le même nombre de points aux cercles intérieurs et extérieurs : le maillage n'est donc pas homogène (Figure 9b). En traçant l'évolution d'une grandeur, par exemple l'énergie en fonction du nombre de points, ou densité n, des pics sont apparus comme sur la Figure 9a.



(a) Évolution de l'énergie en fonction de n le nombre de points sur le cercle intérieur et extérieur. On observe des pics distribués aléatoirement.



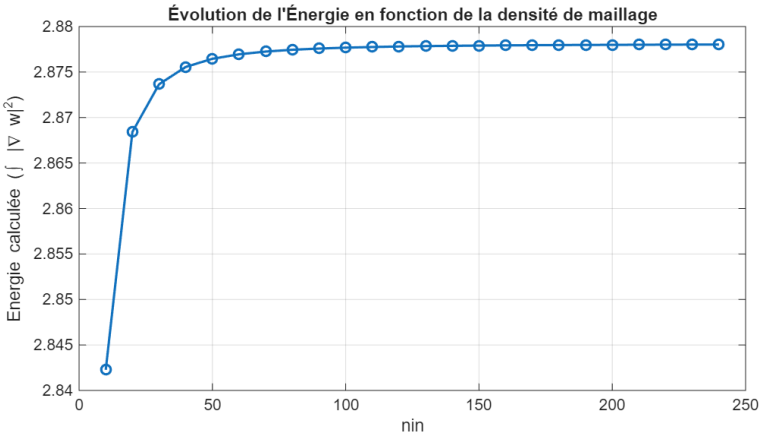
(b) Maillage non homogène

FIGURE 9 – Étude de la convergence pour la configuration pour $\varepsilon = 1$ et $a^\varepsilon = 1.97376432630026e-05$ avec un maillage non homogène.

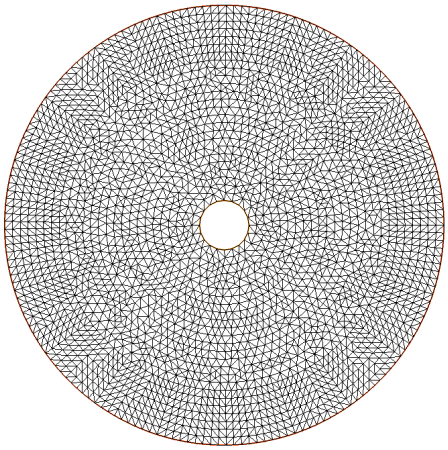
Pour construire un maillage homogène (Figure 10b), on considère que le nombre de points du cercle est proportionnel au rayon, il vient alors :

$$\begin{cases} C\varepsilon = N_{out} \\ Ca^{\varepsilon} = N_{in} \end{cases} \implies C = \frac{N_{out}}{\varepsilon} \implies N_{out} = \frac{\varepsilon}{a^{\varepsilon}} N_{in}$$

On observe ainsi que les pics disparaissent et que la convergence est bien observée Figure 16a.



(a) Évolution de l'énergie en fonction de n le nombre de points sur le cercle intérieur et extérieur. On observe bien la convergence attendue.



(b) Maillage homogène

FIGURE 10 – Étude de la convergence pour la configuration pour pour $\varepsilon = 0.97949$ et $a^{\varepsilon} = 0.11038$ avec un maillage homogène.

3.2.4 Optimisation du maillage et choix des paramètres

La stratégie initiale de maillage, reposant sur une densité de point variable réalisée par FREEFEM via la fonction "Nomdelafonction", s'est avérée peu concluante. L'irrégularité de la taille des triangles empêchait toute comparaison rigoureuse entre les différentes configurations à paramètres ε, a_{ε} et donc C_0 fixés. Pour remédier à ce problème, un second maillage a été réalisé en fixant cette fois-ci la taille caractéristique maximale de triangle (notée h_{max}), garantissant ainsi une homogénéité spatiale de la discrétisation.

Une fois ce maillage implémenté, une seconde étude de convergence a été menée. Toutefois les premiers résultats ont révélé un paradoxe criant : l'erreur relative augmente à mesure que la finesse du maillage croît. Une façon d'expliquer cette divergence apparente réside dans le fait que ce code compare les valeurs obtenues pour les valeurs théorique et numérique en négligeant donc l'erreur intrinsèque liée au fait que ε est ici fixé.

L'augmentation de l'erreur relative lors de l'affinement du maillage peut donc paraître contre-intuitive. Elle peut pourtant s'expliquer par la décomposition de l'erreur totale en deux composantes distinctes :

$$\mathcal{E}_{totale} = \underbrace{\|u_{h,\varepsilon} - u_{\varepsilon}\|}_{\text{Erreur de discrétisation}} + \underbrace{\|u_{\varepsilon} - u_0\|}_{\text{Erreur de modèle}} \tag{14}$$

— **L'erreur de discrétisation** décroît classiquement avec la finesse du maillage ($h \to 0$).

— **L'erreur de modèle** est constante pour un ε fixé ; elle représente l'écart entre le problème local exact et la solution limite asymptotique.

Le phénomène observé de "courbe qui remonte" est le résultat d'une compensation d'erreurs couplées. Pour des maillages grossiers, l'erreur de discrétisation peut compenser numériquement l'erreur de modèle. En affinant le maillage, on réduit l'erreur de discrétisation, ce qui révèle l'erreur de modèle résiduelle. Ce n'est donc pas la précision du calcul qui diminue, mais sa fidélité au modèle intermédiaire u_ε qui augmente, révélant ainsi l'écart structurel avec la limite u_0.

Un changement de méthodologie a alors été proposé : comparer la valeur numérique non plus à la valeur théorique limite mais à celle calculée avant passage à la limite, éliminant ainsi l'erreur due au fait que dans cette configuration ε est faible mais ne tend pas vers 0. Si cette approche améliore significativement les résultats obtenus, elle met aussi nettement en avant une saturation de l'erreur comme la montre la figure ci-dessous. Un calcul parallèle de l'erreur liée à l'intégration numérique permet, en partie d'expliquer ce phénomène.

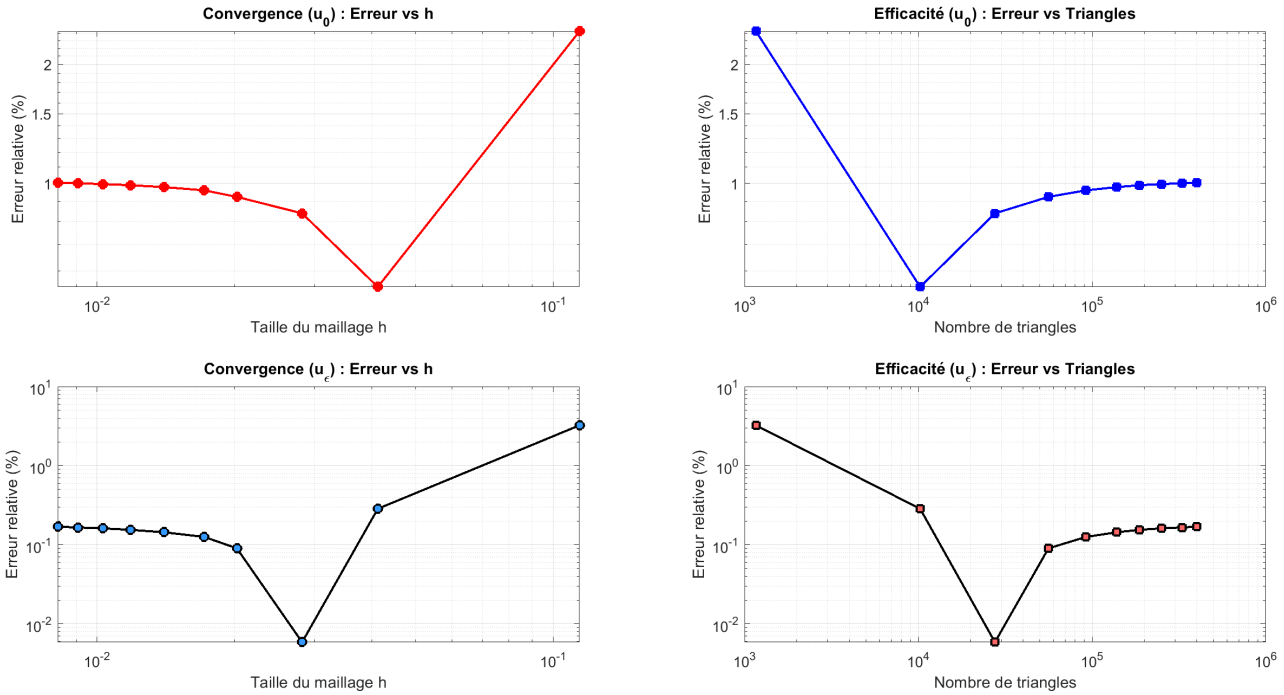


FIGURE 11 – Erreurs relatives en fonctions de la finesse du maillage pour les deux méthodes retenues (avec en haut les graphiques obtenus en comparant les valeurs numériques aux valeurs théoriques asymptotiques et en bas en comparant les valeurs numériques aux valeurs théoriques non asymptotiques)

3.2.5 Comparaison des champs résultants

Une fois la valeur de μ établie, il est possible de remonter à la solution du problème initiale pour finalement comparer non plus simplement les valeurs obtenues pour μ mais surtout les champs obtenus dans les deux cas (problème limite avec les trous et problème homogénéisé sans les trous). A cet effet, le code FREEFEM a été complété pour ajouter les éléments suivants :

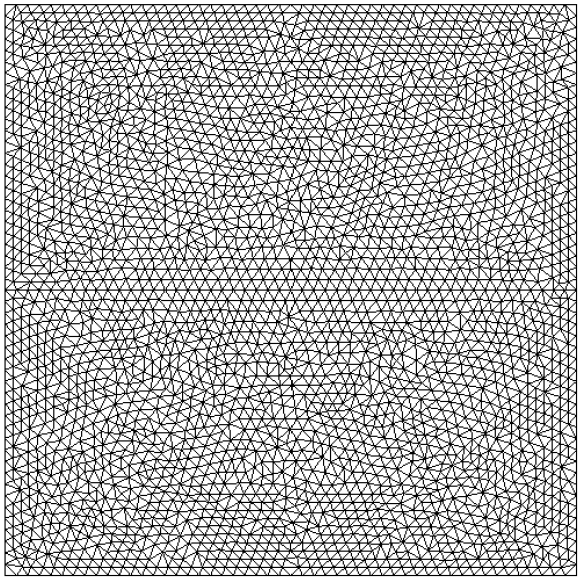
— Calcul des solutions dans le cas avec les trous et le cas homogénéisé.
— Comparaison des deux solutions trouvées en norme L_2.

— Généralisation de la situation précédente en proposant un code effectuant le calcul sur plusieurs cellules.

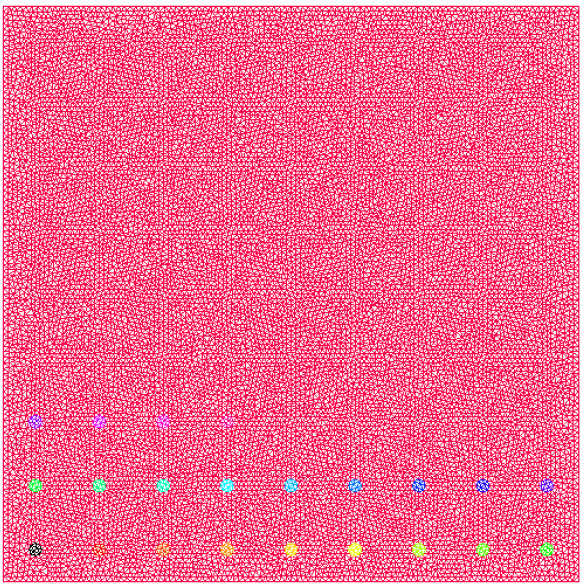
Une fois réalisé, ce code a permis d'obtenir des résultats concluants. En effet, l'erreur relative entre les deux champs obtenus varie globalement entre 25% et 35% selon le terme source f choisi et le nombre de cellules considéré. Le code FreeFEM associé est fourni en annexes.

Cependant à ce stade quelques problèmes ont été identifiés et les corrections associées apportées.

Tout d'abord bien qu'un maillage uniforme avait été concu pour calculer μ proprement, ce maillage n'avait pas été utilisé pour le calcul de la solution dans les deux cas. De plus, une question importante demeurait en suspens, comment calculer la norme L^2 de la différence entre deux fonctions vivant a priori dans deux espaces différents, en effet, u_ε se trouve dans l'espace perforé tandis que u_{hom} vit dans l'espace tout entier. Initialement la méthode choisie était de calculer les solutions sur l'espace perforé, cependant cette démarche introduit un biais dans la comparaison puisque la solution du problème homogénéisé n'est alors pas à proprement dit la solution visée. Pour résoudre ce problème la solution implémentée a été de calculer les solutions sur leurs espaces respectifs, puis de projeter numériquement la solution epsilonesque sur l'espace complet en interpolant sa valeur sur les trous (valeur sensiblement égale à 0). Notons au passage que ce choix requiert donc un maillage plus fin de l'espace tout entier par rapport à l'espace perforé pour que l'interpolation se fasse au mieux. Là encore diverses astuces ont été mises en œuvre pour que l'interpolation soit la plus précise possible, notamment la création d'un maillage prenant en compte la forme des trous malgré leur absence théorique dans le cas homogénéisé, comme le montre la figure 12b.



(a) Maillage homogénéinisé



(b) Maillage homogène avec les trous

Figure 12 – Comparaison des maillages homogénéisés

Un dernier élément à prendre en compte est la définition du domaine, dans le code actuel, la taille du domaine dépend directement de epsilon, or pour que la comparaison soit encore davantage fiable il serait plus pertinent de prendre un domaine de taille fixée et de le subdiviser en cellules toujours plus petites. Une première tentative avait été réalisée mais se heurtait à un certain nombre de problèmes : si le code implémenté réalisait théoriquement l'action demandée les valeurs de a_ε

deviennent très rapidement très petites, typiquement, si l'on étudie les résultats si l'on prend un ε un tout petit peu plus petit (par exemple $\varepsilon = 0.1$ pour avoir `NbCell = 10`, contre $\varepsilon \approx 0.9$ dans le cas précédent) avec $C_0 = 1$:

$$a^\varepsilon = \exp\left(-\frac{1}{0.1^2}\right) = \exp(-100) \approx 3.7 \times 10^{-44}$$

Le a^ε est alors noyé dans l'erreur numérique rendant impossible la résolution du problème en l'état.

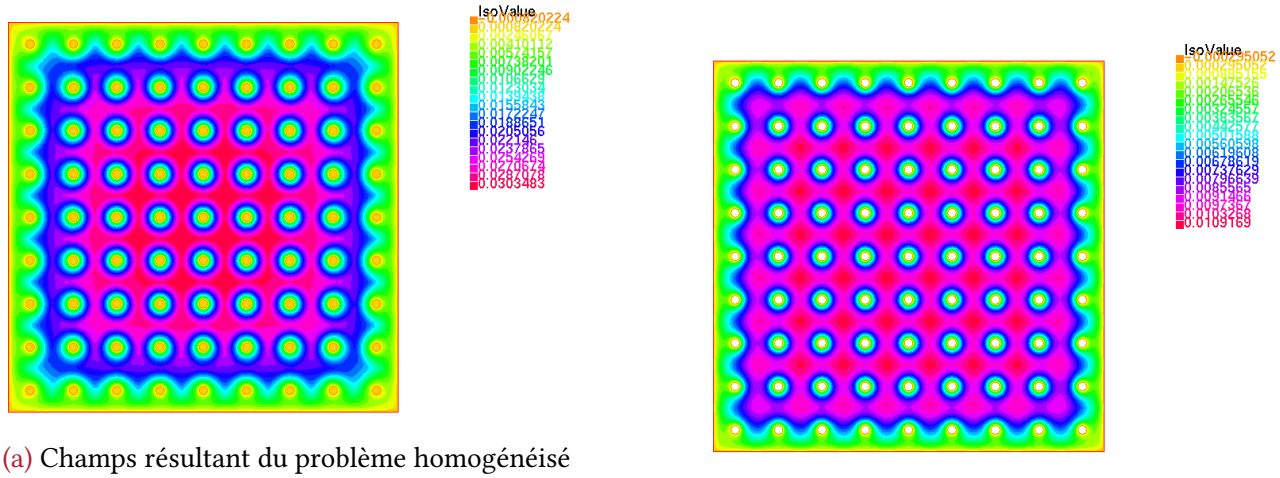
Par ailleurs, notons que cette valeur faible de a^ε pose également problème pour la définition du maillage, en effet, ε étant amené à tendre vers 0, le maillage doit être suffisamment fin notamment autour des trous pour que la résolution soit correcte, cependant la dépendance entre n_{in} et n_{out} retenue jusqu'ici entraîne une valeur de n_{out} très importante et donc du nombre de triangles utilisés pour mailler la surface extrêmement important. Ce problème a cependant été résolu en adaptant le nombre de trous à l'aide d'une nouvelle formule plus adaptée et en utilisant une petite astuce permettant de simplifier les calculs mais pas d'observer la convergence en fonction de ε. Le choix a été fait de fixer un rapport bien précis entre ε et a_ε puis d'en déduire la valeur de C_0 simplifiant par conséquent le problème mais ne permettant que d'obtenir qu'une information à un temps figé.

Enfin il est important de noter que bien que la convergence ait été clairement établie dans les parties précédente, il s'agit d'une convergence dans l'espace H^1, pour laquelle on ne dispose pas d'information concernant la vitesse de convergence. Il est par conséquent difficile d'obtenir une information sur le moment où la convergence est effectivement observable.

Enfin afin d'améliorer les résultats précédents il est possible de comparer non plus u_ε et u mais de comparer u_ε avec $u \times w_\varepsilon$, où la fonction w_ε désigne un filtre incluant les oscillations du milieu et introduit plus tard dans l'article de Cioranescu et Murat, pour lequel la convergence est cette fois-ci en norme L^2.

```
// Extrait : Définition du correcteur w_eps et calcul de l'erreur corrigée
func real weps(real xx, real yy) {
    real val = 1.0;
    for (int i = 0; i < NbTotalHoles; i++) {
        // ... [Calcul des coordonnées (xc, yc) du trou i] ...
        real r = sqrt((xx - xc)^2 + (yy - yc)^2);
        if (r < aEpsilon) { val = 0.0; break; }
        else if (r < epsilon) {
            val = (log(r) - log(aEpsilon)) / (log(epsilon) - log(aEpsilon));
            break;
        }
    }
    return val;
}

// Application du filtre et calcul de la nouvelle différence
Vh3 uCorrected = u3 * weps(x, y);
Vh3 diffCorr = u3interp - uCorrected;
real L2errCorr = sqrt( int2d(Th3)( (diffCorr)^2 ) );
```

Listing 5 – Code FreeFEM : Définition du correcteur et application du filtre



(a) Champs résultant du problème homogénéisé avec filtre

(b) Champs résultant du problème initial

FIGURE 13 – Comparaison des champs calculés

Dans le cas spécifique illustré figure 13, l'erreur obtenue passe de 218% à 152%. Bien que cette diminution soit significative, l'erreur obtenue reste importante du fait des valeurs légèrement différentes calculées sur l'ensemble du domaine. Cependant un point essentiel ici est de noter que les tendances observées dans la solution homogénéisée corrigée sont effectivement les mêmes que dans celles obtenues pour le problème initial.

3.2.6 Cas $N = 3$

L'étude du cas bidimensionnel ayant permis de valider l'approche numérique, plusieurs pistes d'approfondissement étaient envisageables, on peut notamment citer parmi elles l'exploration de géométries singulières en 2D venant remplacer les disques de la partie précédente. Cependant l'extension à la dimension supérieure s'est naturellement imposée comme le prolongement le plus pertinent. En effet, le cas $N = 3$ constitue un pas de plus vers la réalité physique des matériaux et ouvre la voie vers des applications concrètes et ce dans divers domaines, allant de la mécanique, à l'électromagnétisme en passant par la thermique. L'étude tridimensionnelle s'est ainsi articulée autour de deux configurations géométriques distinctes : une forêt de cylindres répartis périodiquement dans le domaine, constituant ainsi un prolongement direct du cas bidimensionnel par simple extrusion, puis un domaine perforé périodiquement par des sphères, dont le traitement numérique a soulevé un certain nombre de défis.

Cas des cylindres

La première configuration tridimensionnelle étudiée est celle de la forêt de cylindres, c'est-à-dire un réseau périodique de cylindres traversant le domaine d'étude. Ce cas se plaçait effectivement dans la suite logique de l'étude bidimensionnelle précédente puisque la géométrie cylindrique consiste en une simple extrusion suivant l'axe vertical z du cas précédent. L'invariance par translation suivant l'axe z permet par ailleurs une comparaison des tendances dans les résultats par rapport au cas des disques $2D$.

D'un point de vue purement numérique, la génération du maillage tridimensionnel et principalement la création des perforations cylindriques peut s'avérer délicate. Les premières tentatives

d'implémentation utilisaient un mailleur volumique classique de FreeFEM : *TetGen*. Néanmoins ces premiers essais ont mis en évidence des problèmes de robustesse, se traduisant entre autres par des erreurs d'orientation des facettes constituant le cylindre ou des défauts géométriques aux frontières des trous. Pour pallier ces difficultés algorithmiques une approche par extrusion a été privilégiée, méthode se plaçant par ailleurs encore davantage dans la continuité du cas précédent que l'alternative *TetGen* car reprend directement le maillage précédent et l'extrude simplement selon l'axe z. La stratégie implémentée dans FreeFEM consiste donc à régénérer dans un premier temps le maillage bidimensionnel étudié initialement, puis l'extruder verticalement sur un nombre de couches préalablement défini à l'aide de la fonction *buildlayer*. Cette méthode a permis de résoudre le problème de robustesse mentionné précédemment en limitant les artefacts géométriques, tout en attribuant automatiquement les labels aux frontières.

```
// Extrait : Extrusion 3D robuste et résolution par gradient conjugué
int nz = 15; // Nombre de couches en Z
int[int] rup = [0, labTop];
int[int] rdown = [0, labBottom];

// Création des cylindres par extrusion du maillage 2D (Th2Micro)
mesh3 Th3Micro = buildlayers(Th2Micro, nz, zbound=[0.0, Lz], labelup=rup,
    labeldown=rdown);

// ... [Définition des espaces P1 3D] ...

// Formulation 3D avec solveur itératif (CG)
solve ProbMicro(uEps, vEps, solver=CG) =
    int3d(Th3Micro)(dx(uEps)*dx(vEps) + dy(uEps)*dy(vEps) + dz(uEps)*dz(vEps
    ))
  - int3d(Th3Micro)(f * vEps)
  + on(labExt, labTop, labBottom, uEps=0)
  + on(labCyl, uEps=0);
```

Listing 6 – Code FreeFEM : Extrusion 3D et formulation du problème 3D

Une fois les problèmes résolus dans les deux configurations : micro (sur le domaine avec les cylindres) et macro (sur le domaine homogénéisé sans les cylindres), la question de l'affichage des solutions s'est posée. Plutôt que de recourir à des logiciel externes de post-traitement, une méthode d'extraction native a été proposée, exploitant ici l'invariance par translation suivant l'axe vertical z. A cet effet, une interpolation de la solution tridimensionnelle sur un plan bidimensionnel (ici situé à mi-hauteur du domaine : $z = L_z/2$) a été effectuée. Cette astuce permet non seulement une comparaison quantitative des champs résultants u_ε et u_{hom} mais également qualitative via une visualisation directe des deux dans un plan donné.

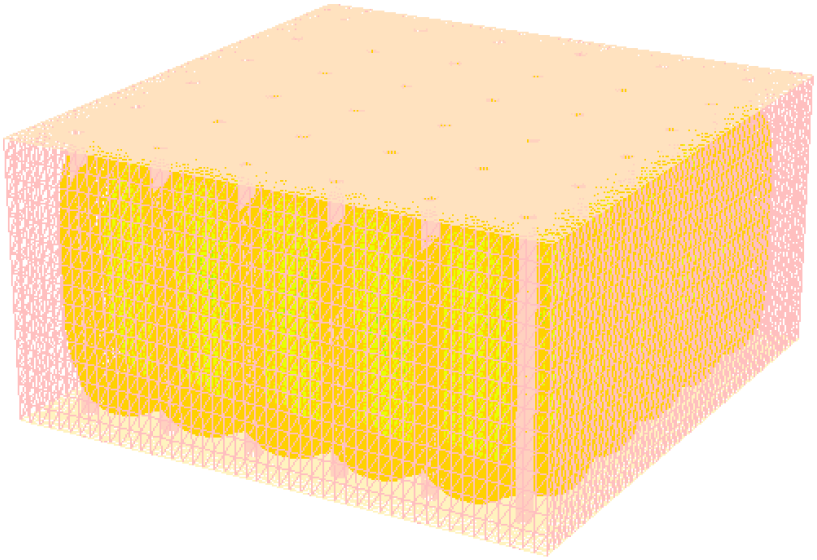
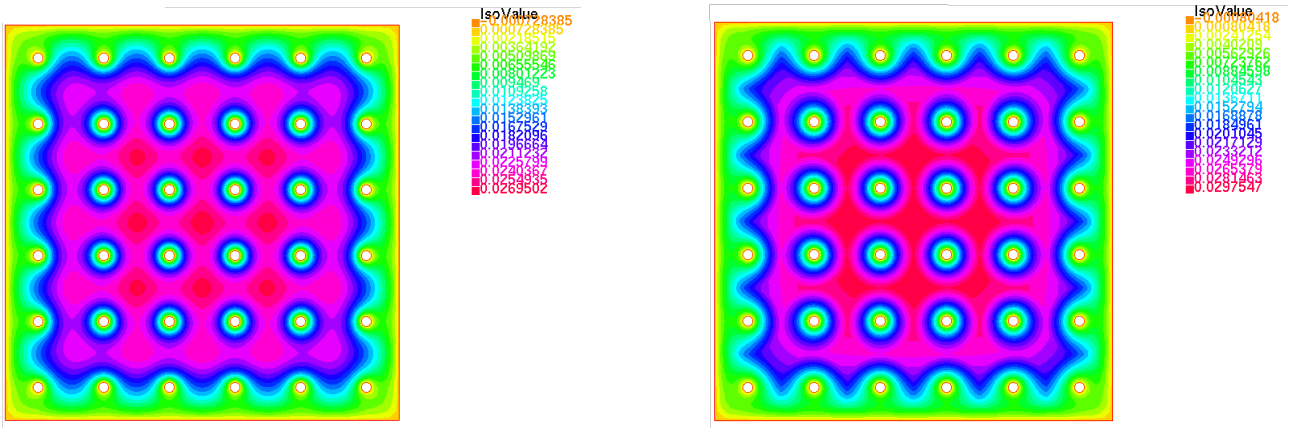


FIGURE 14 – Visualisation du champ obtenu

L'analyse quantitative permet de valider l'approche numérique et notamment la génération du maillage par extrusion. Le calcul de l'erreur relative en norme L^2 sur l'ensemble du domaine révèle un écart de l'ordre de 30%. Cette valeur peut sembler élevée mais est cohérente avec les valeurs trouvées dans l'étude bidimensionnelle. Cette valeur importante peut s'expliquer par les mêmes arguments que ceux avancés dans l'étude bidimensionnelle, à savoir le fait que la convergence soit initialement dans H^1, que l'on ne dispose pas d'information sur la vitesse de convergence et qu'il est difficile d'étudier l'évolution de l'erreur pour un problème, c'est-à-dire à C_0 fixé, tout en diminuant ε du fait que les valeur de a_ε résultantes deviennent rapidement extrêmement faibles (noyées dans le zéros numérique).

Pour améliorer encore davantage les résultats obtenus, il est également possible, comme pour la configuration bidimensionnelle, d'ajouter un correcteur w_ε.



(a) Visualisation de u_ε en $z = L_z/2$

(b) Visualisation de $u_{hom} \times w_\varepsilon$ en $z = L_z/2$

FIGURE 15 – Comparaison des champs obtenus dans le plan en $z = L_z/2$

L'ajout de ce correcteur permet alors de faire chuter l'erreur relative de 30% à 9% démontrant ainsi que le comportement macroscopique du système reste fidèle à la théorie de l'homogénéisation.

Cas des sphères

L'étude des cavités sphériques constitue la dernière partie de notre démarche tridimensionnelle. Si d'un point de vue purement théorique, la résolution de ce problème est identique au cas des cylindres, son implémentation numérique s'est avérée nettement plus complexe que le cas de la forêt cylindrique, exigeant un changement radical de paradigme.

Le premier obstacle majeur a concerné la création du maillage. La génération du maillage avait déjà constitué une étape clé dans le cas précédent et s'est avérée encore plus complexe dans ce cas. En effet, les tentatives de génération de maillage utilisant le mailleur *TetGen* se sont à nouveau soldées par des échecs répétés. Le pavage de surfaces courbes fermées générait, comme pour les cylindres, de forts problèmes d'orientation des normales et, plus grave encore, l'apparition de tétraèdres dégénérés aux pôles des sphères. La présence de ces anomalies provoquait alors des divisions par zéro lors des calculs des gradients produisant ainsi des solutions numériques aberrantes. Par ailleurs, on ne disposait plus désormais de la propriété d'invariance selon l'axe z qui nous avait permis de contourner ce problème en utilisant le maillage 2D que l'on extrudait par la suite.

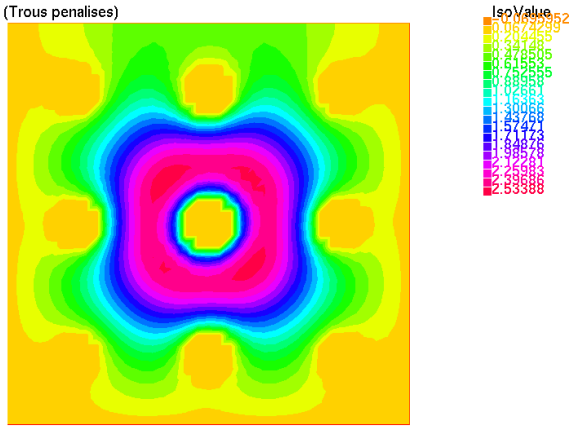
Face à ces problèmes, le choix a finalement été fait d'utiliser une méthode de pénalisation sur domaine fictif. Le principe consiste à abandonner complètement le maillage perforé au profit d'un bloc plein, pour lequel le maillage est extrêmement simple, et de prendre en compte les cavités sphériques non par des frontières physiques, mais par une fonction caractéristique χ valant 1 à l'intérieur des sphères et 0 à l'extérieur. Un terme dit de pénalisation, arbitrairement grand ($10^7 \cdot \chi \cdot u$) a ensuite été ajouté dans la formulation variationnelle afin de forcer la solution u_ε à valoir 0 à l'intérieur des cavités.

De plus une attention particulière à du être portée sur la méthode de résolution choisie. En effet, la résolution se faisant dorénavant sur une grille 3D très dense, le solveur par défaut de FreeFEM (UMFPACK) arrivait rapidement à saturation de la mémoire vive (erreur *Out of Memory*). Une solution possible pour permettre le calcul malgré tout, a été de substituer au solveur par défaut la méthode du gradient conjugué (*solver=CG*), solveur plus adapté aux problèmes faisant intervenir des grandes matrices creuses, symétriques et définies positives comme celles de notre problème.

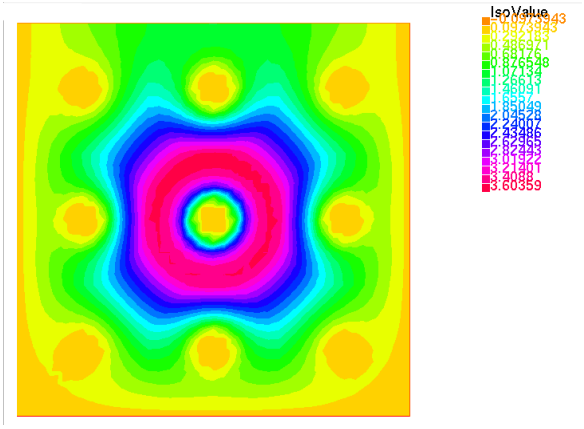
```
// Extrait : Méthode de pénalisation sur domaine fictif (grille dense)

// 1. Définition stricte (binaire) des trous dans l'espace P0
fespace Vh0(Th3, P0);
Vh0 Chi = 0.0;
// ... [Boucle sur i,j,k pour mettre Chi = 1.0 à l'intérieur des sphères]
    ...

// 2. Résolution du problème microscopique avec terme de pénalisation
solve ProbMicro(uEps, vEps, solver=CG) =
    int3d(Th3)(dx(uEps)*dx(vEps) + dy(uEps)*dy(vEps) + dz(uEps)*dz(vEps))
  + int3d(Th3)(1e7 * Chi * uEps * vEps) // Mur de pénalisation
  - int3d(Th3)(f * vEps)
  + on(1, uEps=0);

// ... [Résolution Macro] ...

```

```
// 3. Calcul de l'erreur masquée (exclusion de l'intérieur des trous
   virtuels)
real errL2 = sqrt(int3d(Th3)( (1.0 - Chi) * (uEps - uLim)^2 ));
```

Listing 7 – Code FreeFEM : Méthode de pénalisation



(a) Visualisation du champ u_ε dans la section $z = 0$



(b) Visualisation du champ $u_{hom} \times w_\varepsilon$ dans la section $z = 0$

FIGURE 16 – Comparaison des champs obtenus dans le plan en $z = 0$

La mise en place de la méthode de pénalisation requiert de plus des considérations supplémentaires au moment du calcul de l'erreur relative. En effet, le calcul de la norme L^2 de la différence repose sur l'intégrale du terme $(u_\varepsilon - u_{hom})^2$ sur l'ensemble du domaine. Or ce domaine prend aussi en compte l'intérieur des sphères pénalisées. Un filtre multiplicatif de la forme $(1 - \chi)$ a donc été ajouté pour restreindre le calcul de l'intégrale au domaine privé des sphères uniquement.

Une fois l'ensemble de ces éléments implémentés, l'erreur numérique alors obtenue en norme L^2 est de l'ordre de 59%. A nouveau, cette valeur peut sembler importante dans un premier temps, mais il est important de garder en tête les remarques déjà faites précédemment, à savoir que la convergence théorique est une convergence dans H^1, que l'on ne dispose pas d'information sur la vitesse de convergence et enfin que le problème finalement obtenu diffère tout de même légèrement de celui théorique en raison de la valeur de μ_{num} qui diffère légèrement de μ_{th} et du fait de l'emploi d'une méthode de pénalisation qui modifie tout de même les modalités de l'étude.

Cependant il est encore un fois possible d'améliorer légèrement ce résultat à l'aide d'un correcteur. L'implémentation, dont une visualisation est donnée sur la Figure 16 fait cette fois-ci passer l'erreur relative de 59% à 41%.

3.3 Perspective d'évolution

Si cette étude a permis de répondre à la problématique et au cahier des charges fixé en amont, elle a également mis en lumière certains nouveaux aspects et limites associés à l'application de cette théorie. Tout d'abord comme cela a été mentionné à maintes reprises dans les développements précédents un point crucial qui permettrait de compléter cette étude serait d'obtenir des informations sur la vitesse de convergence. En lien avec cet élément on peut également noter que si la tendance

générale est rendue fidèlement par le modèle, les valeurs diffèrent parfois assez nettement. En conséquence si cette approche numérique permet d'avoir une idée de l'allure générale du champ résultant elle ne rend pas compte fidèlement de la fonction pour ce qui est de la quantification de cette dernière.

Par ailleurs un autre élément intéressant pouvant être traité et qui a été mentionné au moment de l'étude en 3D porterait sur l'étude de formes quelconques qui remplaceraient les trous rendant davantage compte de la réalité physique où les cavités et perforations ne sont jamais parfaitement cylindriques ou sphériques et peuvent relever de formes plus complexes. Ce point, en partie traité par Cioranescu et Murat, du moins du point de vue théorique peut également trouver des applications pratiques, d'où l'intérêt de le simuler numériquement.

Conclusion

Ce projet, encore en cours, représente une opportunité unique de développer nos compétences mathématiques, informatiques et organisationnelles dans la continuité des compétences et connaissances déjà acquises dans notre cursus mathématique et constitue par ailleurs un premier contact avec le travail de chercheur. Les compétences mathématiques associées à ce projet constituent des outils systématiques face à de nombreux problèmes et pourront donc être réutilisés dans des futures études. La gestion de projet repose quant à elle sur une planification rigoureuse et rythmée avec des jalons clairement définis. Ce cadre structuré est essentiel pour mener à bien ce projet.

Ce travail nous a permis d'explorer et d'approfondir les concepts fondamentaux de la théorie de l'homogénéisation et d'identifier avec précision le terme correctif émergent dans la formulation macroscopique à la fois théoriquement et numériquement. La confrontation systématique de ces deux approches a également permis de valider notre approche, mais a aussi permis d'établir clairement une méthodologie rigoureuse pour l'étude de convergence.

Les résultats obtenus à l'issue de ce premier semestre ont permis de confirmer la robustesse du modèle implémenté pour le cas $N = 2$. A présent, plusieurs perspectives s'ouvrent à nous pour prolonger cette étude. L'analyse pourrait ainsi s'étendre à d'autres géométries de perforations, en remplaçant les cercles par d'autres figures, ou s'orienter vers une exploration du cas $N = 3$ correspondant donc à un matériau réel (en trois dimensions), pour confronter la théorie au monde réel, c'est-à-dire à l'étude de domaines volumiques.

[Nouveau paragraphe] Les résultats obtenus pour les configurations bidimensionnelles et tridimensionnelles étudiées ont permis de confirmer la robustesse du modèle numérique implémenté. En revanche, si les tendances sont identiques dans les configurations macro et micro, l'implémentation numérique a tout de même mis en lumière des écarts quantifiés importants principalement associé au problème majeur rencontré : bien que la convergence ait été clairement établie, on ne dispose d'aucune information sur la vitesse de celle-ci rendant son observation complexe et l'application à des cas concrets délicate.

Références

[1] Doina Cioranescu et François Murat. "A Strange Term Coming from Nowhere". In : *Topics in the Mathematical Modelling of Composite Materials*. Sous la dir. d'Andrej Cherkaev et Robert Kohn. Boston, MA : Birkhäuser Boston, 1997, p. 45-93. ISBN : 978-1-4612-2032-9. DOI : 10.1007/978-1-4612-2032-9_4. URL : https://doi.org/10.1007/978-1-4612-2032-9_4.

[2] Haïm Brezis. *ANALYSE FONCTIONNELLE Théorie et applications*. MASSON Paris New York Barcelone Milan Mexico Sao Paulo, 1987.

[3] Matthieu Bonnivard. *Modélisation et analyse mathématique des EDP*. Octobre 2025. URL : https://pedagogie3.ec-lyon.fr/mod/resource/view.php?id=43769.

[4] Filipa Caetano. *Méthode des éléments finis pour les EDP*. 2021-20255. URL : https://www.imo.universite-paris-saclay.fr/~filipa.caetano/EF2021-22.html.

[5] *Logiciels de calcul de structures par 'éléments finis'*. URL : https://pedagogie1.ec-lyon.fr/pluginfile.php/94419/mod_folder/content/0/ansys_Rappel_%20Element_Finis.pdf?forcedownload=1.

[6] Centrale Lyon. *Chapitre II - Les matériaux de construction - Matériaux Composites*. 2025-2026. URL : https://pedagogie2.ec-lyon.fr/mod/folder/view.php?id=54121.

[7] J. D. Eshelby. “The determination of the elastic field of an ellipsoidal inclusion, and related problems”. In : *Proceedings of the Royal Society A : Mathematical, Physical and Engineering Sciences* 241.1226 (1957), p. 376-396. doi : 10.1098/rspa.1957.0133.

Annexes

Présentation des annexes

1) Synthèse complète des résultats mathématiques du Brezis (Annexe 1)

   Cette annexe énumère l'ensemble des informations scientifiques nécessaires à la compréhension du sujet et a été réalisée en rassemblant les propositions et théorèmes formulés à partir du livre *Analyse fonctionnelle* de H. Brezis [2] suivant les conseils de notre tuteur et reprend également les cours : Outils mathématiques avancés pour l'analyse des équations aux dérivées partielles dispensé par Laurent Seppecher à centrale Lyon, ainsi que les cours de Licence 3 suivis à l'université Claude Bernard Lyon 1. Pour les résultats les plus importants, des démonstrations sont également proposées.
2) Présentation complète de la gestion de projet du projet (Annexe 2)

Une partie importante des codes développés sont présents sur le Github associé au projet : https://github.com/MaximeLuce/Un-terme-trange-venu-d-ailleurs.

Codes implémentés

Code MATLAB pour déterminer les triplets

```
clear; clc; close all;

% --- PARAMÈTRES ---
n_points = 10000; % Nombre de points à parcourir pour les valeurs de eps et
    a_eps
eps_vals = logspace(-10, 0, n_points); % Balayage de valeurs potentielles
    pour eps et a_eps
a_vals   = logspace(-10, 0, n_points); % Utilisation d'une échelle
    logarithmique pour atteindre des valeurs faible de ces deux paramètres
K = 100;    % Facteur de condition Z2 > K * Z1
n = 100;    % Paramètre nin (nombre de points sur le cercle intérieur)
N_max = 2000; % Limite pour éviter l'erreur de maillage FreeFEM

% --- CALCULS SUR LA GRILLE ---
[E, A] = meshgrid(eps_vals, a_vals);

% Fonctions
f1 = @(e) -e.^2 .* log(e);
Z1 = f1(E);
Z2 = -E.^2 .* log(A); % Calcul de C0

% --- SELECTION DES TRIPLETS ---
% 1. Condition mathématique : Z2 > K * Z1
mask_math = Z2 > K*Z1;

% 2. Condition numérique : a >= 10^-5
mask_limit = A >= 1e-5;

```

```
% 3. Condition de maillage : n * epsilon / a < 2000
% Cette condition évite l'erreur "Assertion fail : (kkkk++ < 2000)"
mask_mesh = (n .* E ./ A) < N_max;

% Masque Global (Intersection de toutes les conditions)
mask_valid = mask_math & mask_limit & mask_mesh;

% --- EXTRACTION DES TRIPLETS ---
eps_final = E(mask_valid);
a_final   = A(mask_valid);

% Calcul de C0 = epsilon^2 * ln(a_epsilon)
C0_final = (eps_final.^2) .* log(a_final);

% Construction du tableau [Epsilon, a_epsilon, C0]
Resultats = [eps_final, a_final, C0_final];

% --- ÉCHANTILLONNAGE DES RÉSULTATS ---
% Pour plus de lisibilité, seule une partie des résultats répartie uniformé
    ment dans les valeurs sélectionnées est affichée.
pas = 1000; % On prend un point tous les "pas" points trouvés
if size(Resultats, 1) > pas
    Resultats_echantillon = Resultats(1:pas:end, :);
else
    Resultats_echantillon = Resultats; % Pas assez de points pour é
    chantillonner
end

% --- AFFICHAGE ---
fprintf('--- ANALYSE NUMÉRIQUE AVEC ÉCHANTILLONNAGE ---\n');
fprintf('Total points valides : %d\n', size(Resultats, 1));
fprintf('Nombre de points affichés (pas = %d) : %d\n\n', pas, size(
    Resultats_echantillon, 1));

if ~isempty(Resultats_echantillon)
    % Création de la table)
    T = array2table(Resultats_echantillon, 'VariableNames', {'Epsilon', '
    a_epsilon', 'C0'});

    % Optionnel : on rajoute le calcul de nout pour vérifier la condition de
     maillage
    T.Nout_FreeFEM = ceil(n .* T.Epsilon ./ T.a_epsilon);

    disp(T);
else
    fprintf('AUCUN POINT TROUVÉ : Les contraintes sont trop restrictives.\n'
    );
end
```

Listing 8 – Script : triplets gagnants

Code FreeFEM final pour le cas 2D

```
// --- 1. Paramètres du problème ---

real L = 1;
int NbCell = 9;

real epsilon = L/NbCell;
//real C0 = 0.1;
//real aEpsilon = exp(-C0/epsilon^2);
real q=0.2;
real aEpsilon = q*epsilon;
real C0 = -epsilon^2*log(aEpsilon);

cout << "--- Parametres ---" << endl;
cout << "Epsilon   = " << epsilon << endl;
cout << "NbCellules= " << NbCell << "x" << NbCell << endl;
cout << "a^epsilon = " << aEpsilon << endl;
cout << "C0        = " << C0 << endl;

// --- 2. Paramètres de Maillage ---
int labSquare = 1;
int labHole = 3;

// Densité
int nout1=50;
int nout=100;
int nin = nout*aEpsilon*4;

cout << "--- Densite de trous ---" << endl;
cout << "Nin   = " << nin << endl;
cout << "Nout   = " << nout << endl;

// --- 3. Définition des Frontières ---

// 3.1 Le Carré extérieur (S'adapte automatiquement à NbCell)
// Le domaine va de -L à +L (puisque la maille fait 2*epsilon)
border Square1(t=-L, L) { x = t; y = -L; label = labSquare; }
border Square2(t=-L, L) { x = L; y = t; label = labSquare; }
border Square3(t=L, -L) { x = t; y = L; label = labSquare; }
border Square4(t=L, -L) { x = -L; y = t; label = labSquare; }

// 3.2 Les Trous (Généralisés) [cite: 1]
// Syntaxe : border Nom(t=0, 2*pi; i)
// 'i' est l'indice du trou, allant de 0 à (NbTotal - 1)
int NbTotalHoles = NbCell * NbCell;

border Holes(t=0, 2*pi; i) {
    // On convertit l'indice linéaire 'i' en indices de grille (ix, iy)
    int iy = i / NbCell;
    int ix = i % NbCell;

    // Calcul du centre du trou (xc, yc)
```

```
    // On centre le réseau autour de (0,0)
    // Le pas entre deux trous est 2*epsilon
    real xc = (ix - (NbCell-1)/2.0) * 2 * epsilon;
    real yc = (iy - (NbCell-1)/2.0) * 2 * epsilon;

    x = xc + aEpsilon * cos(t);
    y = yc + aEpsilon * sin(t);
    label = labHole;
}

// --- 4. Construction du Maillage ---

// Astuce : Pour mailler une frontière paramétrée 'Holes',
// il faut passer un vecteur d'entiers définissant la discrétisation pour chaque 'i'.
int[int] nHoles(NbTotalHoles);
nHoles = -nin; // On met -nin pour tous les trous (orientation horaire pour les trous)

mesh Th = buildmesh(
    Square1(nout1) + Square2(nout1) + Square3(nout1) + Square4(nout1) //+
    //Holes(nHoles) // FreeFEM comprend qu'il doit générer tous les trous définis par le vecteur
);

mesh Th1 = buildmesh(
    Square1(nout) + Square2(nout) + Square3(nout) + Square4(nout) +
    Holes(nHoles) // FreeFEM comprend qu'il doit générer tous les trous définis par le vecteur
);

// Visualisation du maillage pour vérifier
//plot(Th, wait=true, cmm="Maillage avec " + NbTotalHoles + " trous");

plot(Th, cmm="Maillage sans trous", wait=true);
plot(Th1, cmm="Maillage avec trous", wait=true);

// --- 5. Espaces et Variables (Le reste est identique) ---
fespace Vh(Th, P1);
Vh u2, v2;
//real mu = (pi) / (2 * C0+log(epsilon)*epsilon^2);
real mu = (pi) / (2 * C0);

cout << "mu   = " << mu << endl;

//func f = (x^2+y^2 < (L/1.5)^2) ? 10 : 0; // Exemple de source centrée (optionnel, j'ai changé f pour éviter les débordements exponentiels si L est grand)
// Note : J'ai changé 'f' car votre ancienne f = exp(-(x^2+y^2)) devient très petite si on augmente la taille du domaine.
// Remettez votre f d'origine si vous préférez :
//func fOriginal =  exp( - (x + y)^2 );
func fOriginal =  1;
```

```
fespace Vh1(Th1, P1);
Vh1 u1, v1;

// --- 6. Résolutions ---

// Problème 1 : Poisson avec trous
solve Poisson(u1, v1) =
    int2d(Th1)( dx(u1)*dx(v1) + dy(u1)*dy(v1) )
  - int2d(Th1)( fOriginal * v1 )
  + on(labHole, u1=0)
  + on(labSquare, u1=0);

// Problème 2 : Homogénéisé
solve Helmholtz(u2, v2) =
    int2d(Th)( dx(u2)*dx(v2) + dy(u2)*dy(v2) + mu*u2*v2 )
  - int2d(Th)( fOriginal * v2 )
  + on(labSquare, u2=0); // CL Dirichlet homogène sur le bord extérieur

Vh u1interp=u1;

int NbTotalHoles3=3;

int[int] nHoles3(NbTotalHoles);
nHoles3 = nin; // On met -nin pour tous les trous (orientation horaire pour
    les trous)

mesh Th3 = buildmesh(
    Square1(nout) + Square2(nout) + Square3(nout) + Square4(nout) +
    Holes(nHoles3) // FreeFEM comprend qu'il doit générer tous les trous dé
    finis par le vecteur
);

fespace Vh3(Th3,P1);
Vh3 u3, v3;

// Problème 2 : Homogénéisé
solve Helmholtz3(u3, v3) =
    int2d(Th3)( dx(u3)*dx(v3) + dy(u3)*dy(v3) + mu*u3*v3 )
  - int2d(Th3)( fOriginal * v3 )
  + on(labSquare, u3=0); // CL Dirichlet homogène sur le bord extérieur

Vh3 u3interp=u1;

Vh3 diff3 = u3interp - u3;
real L2error3 = sqrt( int2d(Th3)( (diff3)^2 ) );
real L2normU13 = sqrt( int2d(Th3)( (u3interp)^2 ) );

plot(Th3, cmm="Maillage sans trous", wait=true);
plot(u3interp, value=true, fill=true, cmm="u1interp", wait=true);

cout << "-------------------------------------" << endl;
cout << "Erreur relative L2 = " << (L2error3 / L2normU13)*100 << " %" <<
```

```
    endl;
cout << "-------------------------------------" << endl;

// --- Création de la fonction w_eps ---

func real weps(real xx, real yy) {
    real val = 1.0; // Par défaut, on est hors des zones d'influence

    // On itère sur la grille des centres (même logique que pour les
    bordures)
    for (int i = 0; i < NbTotalHoles; i++) {
        int iy = i / NbCell;
        int ix = i % NbCell;

        real xc = (ix - (NbCell-1)/2.0) * 2.0 * epsilon;
        real yc = (iy - (NbCell-1)/2.0) * 2.0 * epsilon;

        real r = sqrt((xx - xc)^2 + (yy - yc)^2);

        if (r < aEpsilon) {
            val = 0.0; // Dans le trou
            break;     // On a trouvé le trou, on sort de la boucle
        }
        else if (r < epsilon) {
            val = (log(r) - log(aEpsilon)) / (log(epsilon) - log(aEpsilon));
            break;
        }
    }
    return val;
}

Vh3 wPlot = weps(x, y);
plot(wPlot, fill=true, value=true, cmm="Correcteur w_epsilon", wait=true);

// Nouvelle fonction pour comparer

// On crée le produit u_homogénéisé * correcteur
Vh3 uCorrected = u3 * weps(x, y);

// Calcul de la différence
Vh3 diffCorr = u3interp - uCorrected;

// Calcul des normes L2
real L2errCorr = sqrt( int2d(Th3)( (diffCorr)^2 ) );
real L2normExact = sqrt( int2d(Th3)( (u3interp)^2 ) );

real erreurRelative = (L2errCorr / L2normExact) * 100;

cout << "--- Comparaison avec Correcteur ---" << endl;
cout << "Erreur relative L2 (avec w_eps) = " << erreurRelative << " %" <<
    endl;
cout << "-----------------------------------" << endl;

```

```
// Visualisation pour vérifier la "gueule" de la solution corrigée
plot(uCorrected, fill=true, value=true, cmm="Produit u3 * w_eps", wait=true)
    ;

// --- 7. Résultats ---
cout << "Max u1 = " << u1[].max << endl;
cout << "Max u2 = " << u2[].max << endl;

Vh diff = u1interp - u2;
real L2error = sqrt( int2d(Th)( (diff)^2 ) );
real L2normU1 = sqrt( int2d(Th)( (u1interp)^2 ) );

plot(u1interp, value=true, fill=true, cmm="u1interp", wait=true);

plot(u1, value=true, fill=true, cmm="U_epsilon (Poisson)", wait=true);
plot(u2, value=true, fill=true, cmm="U_limite (Helmholtz)", wait=true);
plot(diff, value=true, fill=true, cmm="Difference", wait=true);
plot(Th, cmm="Maillage sans trous", wait=true);
plot(Th1, cmm="Maillage avec trous", wait=true);

cout << "-------------------------------------" << endl;
cout << "Erreur relative L2 = " << (L2error / L2normU1)*100 << " %" << endl;
cout << "-------------------------------------" << endl;
```

Listing 9 – Code FreeFEM : 2D

Code FreeFEM final pour les cylindres

```
// ====================================================================
//    HOMOGENEISATION 3D - FORET DE CYLINDRES - EXEMPLE 2.12
//    SANS logiciel externe (Affichage natif via coupes 2D)
// ====================================================================

load "msh3" // Module 3D natif

// --- 1. PARAMETRES ---
int NbCell = 6;         // Nombre de cellules dans le plan XY
real L = 1.0;           // Demi-longueur XY
real Lz = 1.0;          // Hauteur des cylindres

real epsilon = L / NbCell;
real rayonRatio = 0.15;
real aEpsilon = rayonRatio * epsilon;

// Formule "Pré-asymptotique" pour mu (Capacité exacte de la cellule finie)
real mu = pi / ( 2.0 * (epsilon^2) * log(epsilon / aEpsilon) );

cout << "--- Parametres Foret de Cylindres 3D ---" << endl;
cout << "Nb Cylindres : " << NbCell*NbCell << endl;
cout << "Epsilon      : " << epsilon << endl;
cout << "Mu calcule   : " << mu << endl;

// ---> LES FAMEUX LABELS SONT ICI <---
int labExt = 1;       // Parois latérales extérieures
int labCyl = 2;       // Parois des cylindres (trous)
int labTop = 3;       // Plafond 3D
int labBottom = 4;    // Plancher 3D

// --- 2. CONSTRUCTION DU MAILLAGE 2D DE BASE ---
// On crée d'abord la "base" trouée en 2D
int nPointsExt = NbCell * 8;
int nPointsHole = 15;

border S1(t=-L, L){x=t; y=-L; label=labExt;}
border S2(t=-L, L){x=L; y=t; label=labExt;}
border S3(t=L, -L){x=t; y=L; label=labExt;}
border S4(t=L, -L){x=-L; y=t; label=labExt;}

int NbHoles = NbCell * NbCell;
border Holes(t=0, 2*pi; i) {
    int iy = i / NbCell;
    int ix = i % NbCell;
    real xc = -L + epsilon + ix * (2*epsilon);
    real yc = -L + epsilon + iy * (2*epsilon);
    x = xc + aEpsilon * cos(t);
    y = yc + aEpsilon * sin(t);
    label = labCyl;
}

```

```
int[int] nHolesVec(NbHoles);
nHolesVec = -nPointsHole;

// Maillages 2D de base
mesh Th2Micro = buildmesh(S1(nPointsExt) + S2(nPointsExt) + S3(nPointsExt) +
    S4(nPointsExt) + Holes(nHolesVec));
mesh Th2Macro = buildmesh(S1(nPointsExt) + S2(nPointsExt) + S3(nPointsExt) +
    S4(nPointsExt));

// --- 3. EXTRUSION 3D (CREATION DES CYLINDRES) ---
// On utilise buildlayers qui extrude le 2D en 3D (très robuste, pas besoin
   de TetGen)
int nz = 15; // Nombre de couches en Z

cout << "Extrusion de la foret de cylindres en 3D..." << endl;

// CORRECTION : Création des tableaux de correspondance pour les labels Haut
    et Bas
// On dit à FreeFEM : "Prends l'intérieur du maillage (label 0) et
   transforme-le"
int[int] rup = [0, labTop];
int[int] rdown = [0, labBottom];

// CORRECTION : zbound doit contenir des réels (0.0 au lieu de 0)
mesh3 Th3Micro = buildlayers(Th2Micro, nz, zbound=[0.0, Lz], labelup=rup,
   labeldown=rdown);
mesh3 Th3Macro = buildlayers(Th2Macro, nz, zbound=[0.0, Lz], labelup=rup,
   labeldown=rdown);

// --- 4. PROBLEMES 3D ---
fespace Vh3Micro(Th3Micro, P1);
Vh3Micro uEps, vEps;

fespace Vh3Macro(Th3Macro, P1);
Vh3Macro uLim, vLim;

// Source thermique purement 3D : Une boule de chaleur au centre du volume
func f = 100 * exp(-8*(x^2 + y^2 + (z-Lz/2)^2));

cout << "Resolution du modele Micro (vrais cylindres)..." << endl;
solve ProbMicro(uEps, vEps, solver=CG) = // <--- AJOUTER ICI
    int3d(Th3Micro)(dx(uEps)*dx(vEps) + dy(uEps)*dy(vEps) + dz(uEps)*dz(vEps
   ))
  - int3d(Th3Micro)(f * vEps)
  + on(labExt, labTop, labBottom, uEps=0)
  + on(labCyl, uEps=0);

cout << "Resolution du modele Macro (terme etrange)..." << endl;
solve ProbMacro(uLim, vLim, solver=CG) = // <--- AJOUTER ICI
    int3d(Th3Macro)(dx(uLim)*dx(vLim) + dy(uLim)*dy(vLim) + dz(uLim)*dz(vLim
   )
                    + mu * uLim * vLim)
  - int3d(Th3Macro)(f * vLim)
```

```
  + on(labExt, labTop, labBottom, uLim=0);


// --- 5. VISUALISATION NATIVE (ZÉRO LOGICIEL EXTERNE) ---
// On extrait la solution 3D sur le plan 2D situé à mi-hauteur (z = Lz/2)
cout << "Extraction des coupes 2D pour affichage..." << endl;

fespace Vh2Micro(Th2Micro, P1);
Vh2Micro uEpsCoupe;
uEpsCoupe = uEps(x, y, Lz/2); // Magie FreeFEM : Interpolation 3D -> 2D !

fespace Vh2Macro(Th2Macro, P1);
Vh2Macro uLimCoupe;
uLimCoupe = uLim(x, y, Lz/2);

// On interpole la solution Macro sur le maillage Micro pour pouvoir faire
    la soustraction
Vh2Micro uLimInterp2D = uLimCoupe;
Vh2Micro diffCoupe = uEpsCoupe - uLimInterp2D;

// Affichage classique 2D natif
plot(uEpsCoupe, fill=1, value=1, cmm="Coupe a mi-hauteur: Micro (Vrais
    Cylindres)", wait=1);
plot(uLimCoupe, fill=1, value=1, cmm="Coupe a mi-hauteur: Macro (
    Homogeneisee)", wait=1);
plot(diffCoupe, fill=1, value=1, cmm="Difference sur la coupe", wait=1);

cout << "Calcul de l'erreur..." << endl;

// Calcul de l'erreur sur le volume 3D
Vh3Micro uLimInterp3D = uLim;
Vh3Micro diff3D = uEps - uLimInterp3D;
real errL23D = sqrt(int3d(Th3Micro)(diff3D^2));
real norm3D = sqrt(int3d(Th3Micro)(uEps^2));

cout << "-------------------------------------" << endl;
cout << "Erreur Relative L2 sur tout le volume 3D : " << (errL23D/norm3D)
    *100 << " %" << endl;
cout << "-------------------------------------" << endl;



// ====================================================================
// --- 6. CORRECTEUR W_EPS ET NOUVELLE ERREUR 3D ---
// ====================================================================

cout << "Calcul et application du correcteur w_eps..." << endl;

func real weps3D(real xx, real yy, real zz) {
    real val = 1.0;

    for (int i = 0; i < NbHoles; i++) {
        int iy = i / NbCell;
```

```
        int ix = i % NbCell;

        // Centres exacts utilisés pour la bordure 'Holes'
        real xc = -L + epsilon + ix * (2*epsilon);
        real yc = -L + epsilon + iy * (2*epsilon);

        // L'astuce est ici : on ne prend pas en compte zz
        // car on calcule la distance à l'axe vertical du cylindre !
        real r = sqrt((xx - xc)^2 + (yy - yc)^2);

        if (r < aEpsilon) {
            val = 0.0; // Dans le cylindre
            break;
        }
        else if (r < epsilon) {
            // Dans la zone d'influence du cylindre
            val = (log(r) - log(aEpsilon)) / (log(epsilon) - log(aEpsilon));
            break;
        }
    }
    return val;
}

// 1. Application du correcteur sur le maillage volumique 3D
// uLimInterp3D a déjà été calculée dans votre code précédent
Vh3Micro uCorrected3D = uLimInterp3D * weps3D(x, y, z);

// 2. Calcul de la nouvelle différence
Vh3Micro diffCorr3D = uEps - uCorrected3D;

// 3. Calcul de la norme et de l'erreur
real errL2Corr3D = sqrt(int3d(Th3Micro)(diffCorr3D^2));

cout << "=====================================" << endl;
cout << " BILAN DES ERREURS 3D (Cylindres)" << endl;
cout << "=====================================" << endl;
cout << "Erreur Relative L2 (SANS correcteur) : " << (errL23D/norm3D)*100 <<
     " %" << endl;
cout << "Erreur Relative L2 (AVEC correcteur) : " << (errL2Corr3D/norm3D)
    *100 << " %" << endl;
cout << "=====================================" << endl;


// --- Visualisation du bénéfice sur la coupe 2D ---
// On applique le correcteur sur la coupe à z = Lz/2
Vh2Micro uCorrectedCoupe = uLimInterp2D * weps3D(x, y, Lz/2);
Vh2Micro diffCorrCoupe = uEpsCoupe - uCorrectedCoupe;

plot(uCorrectedCoupe, fill=1, value=1, cmm="Coupe a mi-hauteur: Macro
    CORRIGEE (uLim * w_eps)", wait=1);
plot(diffCorrCoupe, fill=1, value=1, cmm="NOUVELLE Difference (Avec
```

```
correcteur)", wait=1);
```

Listing 10 – Code FreeFEM : cylindres

Code FreeFEM final pour les sphères

```
// ==================================================================
//    HOMOGENEISATION 3D - SPHERES (PÉNALISATION SUR GRILLE DENSE)
//    L'approche la plus robuste pour la 3D
// ==================================================================

load "msh3"

// --- 1. PARAMETRES ---
int NbCell = 3;
real L = 1.0;
real epsilon = L / NbCell;

// On prend des sphères bien visibles pour que la grille les capte
real rayonRatio = 0.3;
real aEpsilon = rayonRatio * epsilon;

// Formule de mu asymptotique pour la 3D (Capacité sphérique sur volume
    cellule)
real mu = (pi * aEpsilon) / (2.0 * epsilon^3);

cout << "--- Parametres ---" << endl;
cout << "Epsilon      : " << epsilon << endl;
cout << "Rayon aEps   : " << aEpsilon << endl;
cout << "Mu calcule   : " << mu << endl;

// --- 2. MAILLAGE 3D REGULIER (ZÉRO TETGEN) ---
// On crée une grille très fine (36x36x36).
// Chaque "voxel" sera plus petit que la sphère, assurant une forme correcte
    .
int nBox = 36;

// On unifie tous les bords 2D sous le label 1
int[int] labBorders = [1,1, 2,1, 3,1, 4,1];
mesh Th2Box = square(nBox, nBox, [2*L*x-L, 2*L*y-L], label=labBorders);

cout << "Generation de la grille 3D dense..." << endl;
// Extrusion pour faire un cube parfait. Les faces Z- et Z+ prennent le
    label 1.
int[int] rZ = [0, 1];
mesh3 Th3 = buildlayers(Th2Box, nBox, zbound=[-L, L], labelup=rZ, labeldown=
    rZ);

// --- 3. DEFINITION DES TROUS EXACTE (P0) ---
fespace Vh0(Th3, P0);
Vh0 Chi = 0.0;

cout << "Echantillonnage des spheres..." << endl;
for(int i=0; i<NbCell; i++){
    for(int j=0; j<NbCell; j++){
        for(int k=0; k<NbCell; k++){
```

```
            real xc = -L + epsilon + i*(2*epsilon);
            real yc = -L + epsilon + j*(2*epsilon);
            real zc = -L + epsilon + k*(2*epsilon);
            Chi = Chi + ( ((x-xc)^2 + (y-yc)^2 + (z-zc)^2 <= aEpsilon^2) ? 1.0 : 0.0 );
        }
    }
}

// --- 4. RESOLUTION ---
fespace Vh3(Th3, P1);
Vh3 uEps, vEps;
Vh3 uLim, vLim;

func f = 100 * exp(-5*(x^2 + y^2 + z^2));

// On remet solver=CG car la grille est énorme (> 50 000 noeuds) mais parfaitement régulière !
cout << "Resolution Micro (CG)..." << endl;
solve ProbMicro(uEps, vEps, solver=CG) =
    int3d(Th3)(dx(uEps)*dx(vEps) + dy(uEps)*dy(vEps) + dz(uEps)*dz(vEps))
  + int3d(Th3)(1e7 * Chi * uEps * vEps) // Mur de penalisation
  - int3d(Th3)(f * vEps)
  + on(1, uEps=0); // Label 1 couvre toute la boite

cout << "Resolution Macro (CG)..." << endl;
solve ProbMacro(uLim, vLim, solver=CG) =
    int3d(Th3)(dx(uLim)*dx(vLim) + dy(uLim)*dy(vLim) + dz(uLim)*dz(vLim))
  + int3d(Th3)(mu * uLim * vLim)
  - int3d(Th3)(f * vLim)
  + on(1, uLim=0);


// --- 5. ANALYSE DES RESULTATS ---
cout << "=====================================" << endl;
cout << "Max uEps (Avec vrais trous) : " << uEps[].max << endl;
cout << "Max uLim (Modele Lisse)     : " << uLim[].max << endl;

real errL2 = sqrt(int3d(Th3)( (1.0 - Chi) * (uEps - uLim)^2 ));
real normRef = sqrt(int3d(Th3)( (1.0 - Chi) * (uEps)^2 ));

cout << "-------------------------------------" << endl;
cout << "Erreur Relative L2 Volume 3D : " << (errL2/normRef)*100 << " %" << endl;
cout << "=====================================" << endl;

// Visualisation 2D à mi-hauteur
mesh Th2Slice = square(nBox, nBox, [2*L*x-L, 2*L*y-L]);
fespace Vh2(Th2Slice, P1);

Vh2 uEpsCoupe = uEps(x, y, 0.0);
Vh2 uLimCoupe = uLim(x, y, 0.0);
Vh2 diffCoupe = uEpsCoupe - uLimCoupe;
```

```
plot(uEpsCoupe, fill=1, value=1, cmm="Coupe a Z=0 : Micro (Trous penalises)"
    , wait=1);
plot(uLimCoupe, fill=1, value=1, cmm="Coupe a Z=0 : Macro", wait=1);
plot(diffCoupe, fill=1, value=1, cmm="Difference", wait=1);


// ====================================================================
// --- 6. CORRECTEUR W_EPS SPHÉRIQUE ET NOUVELLE ERREUR 3D ---
// ====================================================================

cout << "Calcul et application du correcteur spherique w_eps..." << endl;

int NbTotalSpheres = NbCell * NbCell * NbCell;

func real wepsSphere(real xx, real yy, real zz) {
    real val = 1.0;

    for (int idx = 0; idx < NbTotalSpheres; idx++) {
        // Décodage de l'indice 1D vers 3D (i, j, k)
        int k = idx / (NbCell * NbCell);
        int reste = idx % (NbCell * NbCell);
        int j = reste / NbCell;
        int i = reste % NbCell;

        real xc = -L + epsilon + i*(2*epsilon);
        real yc = -L + epsilon + j*(2*epsilon);
        real zc = -L + epsilon + k*(2*epsilon);

        // Vraie distance 3D
        real r = sqrt((xx - xc)^2 + (yy - yc)^2 + (zz - zc)^2);

        if (r < aEpsilon) {
            val = 0.0; // Dans la sphère
            break;
        }
        else if (r < epsilon) {
            // Formule 3D (Laplacien sphérique en 1/r)
            val = (1.0/aEpsilon - 1.0/r) / (1.0/aEpsilon - 1.0/epsilon);
            break;
        }
    }
    return val;
}

// 1. Application du correcteur sur le champ macro
Vh3 uCorrected3D = uLim * wepsSphere(x, y, z);

// 2. Calcul de l'erreur avec le correcteur
// Note : Le masque (1.0 - Chi) est conservé pour être rigoureux vis-à-vis
    du domaine
real errL2Corr = sqrt(int3d(Th3)( (1.0 - Chi) * (uEps - uCorrected3D)^2 ));

```

```
cout << "=====================================" << endl;
cout << " BILAN DES ERREURS 3D (Spheres)" << endl; // <--- Plus d'accent ici
cout << "=====================================" << endl;
cout << "Erreur Relative L2 (SANS correcteur) : " << (errL2/normRef)*100 <<
    " %" << endl;
cout << "Erreur Relative L2 (AVEC correcteur) : " << (errL2Corr/normRef)*100
     << " %" << endl;
cout << "=====================================" << endl;

// --- Visualisation du benefice sur la coupe Z=0 ---
Vh2 uCorrectedCoupe = uLimCoupe * wepsSphere(x, y, 0.0);
Vh2 diffCorrCoupe = uEpsCoupe - uCorrectedCoupe;

plot(uCorrectedCoupe, fill=1, value=1, cmm="Coupe Z=0: Macro CORRIGEE", wait
    =1);
plot(diffCorrCoupe, fill=1, value=1, cmm="NOUVELLE Difference", wait=1);
```

Listing 11 – Code FreeFEM : sphères